



UNIVERSITY OF PIRAEUS
SCHOOL OF INFORMATION TECHNOLOGIES
DEPARTMENT OF INFORMATICS

Thesis

Thesis Title:	Development of a 3D Sudoku Cube Game for the Web: Algorithm Implementation and Interactive Visualization using Three.js
Τίτλος Πτυχιακής Εργασίας:	Ανάπτυξη Τρισδιάστατου (3D) Παιχνιδιού Sudoku Κύβου (Sudoku Cube) στο Web: Υλοποίηση Αλγορίθμου και Διαδραστική Απεικόνιση με Three.js
Student's name-surname:	Giannakopoulos Nikolaos Ioannis
Όνοματεπώνυμο φοιτητή:	Γιαννακόπουλος Νικόλαος Ιωάννης
Father's name:	Stavros
Όνομα Πατρός:	Σταύρος
Student's ID No:	Π/ 22029
Αριθμός Μητρώου:	
Supervisor:	Efthimios Alepis, Professor Ευθύμιος Αλέπης,
Επιβλέπων:	Καθηγητής

Copyright ©

It is prohibited to copy, store, and distribute this work, in whole or in part, for commercial purposes. Reprinting, storing, and distributing for non-profit, educational, or research purposes is permitted, provided that the source is acknowledged and this message is retained.

The views and conclusions contained in this document solely reflect the author's opinions and do not represent the official positions of the University of Piraeus.

As the author of this work, I declare that this work does not constitute a product of plagiarism and does not contain material from unacknowledged sources.

Acknowledgments

I extend my deepest thanks to my family for their endless support and encouragement during my academic pursuits. Their steadfast love, patience, and confidence in my potential kept me continually motivated, helping me navigate every high and low. I am truly grateful for the sacrifices they made and for their reliable presence whenever I needed them.

Additionally, I offer my genuine thanks to my professors for their exceptional mentorship and direction. Their deep knowledge and passion for education have significantly deepened my grasp of this field and pushed me to reach my highest potential. I am especially appreciative of the insightful feedback and encouragement that directly shaped this thesis. Thank you for your faith in my abilities and your dedication to my educational development.

Περίληψη

Η παρούσα πτυχιακή εργασία στοχεύει τον σχεδιασμό, την ανάπτυξη και την τεχνική τεκμηρίωση της εφαρμογής **CubeDoku (3D Sudoku Game)**, ενός καινοτόμου παιχνιδιού λογικής που μεταφέρει την κλασσική εμπειρία του Sudoku σε έναν τρισδιάστατο κύβο. Η εφαρμογή αποτελείται από ένα frontend βασισμένο σε React με τρισδιάστατη απεικόνιση μέσω Three.js και ένα backend σε ASP.NET Core 8 με βάση δεδομένων PostgreSQL (Neon).

Το CubeDoku εισάγει δύο πρωτότυπους κανόνες πέρα του κλασσικού Sudoku: τον κανόνα ακμής (δύο κελιά που μοιράζονται φυσική ακμή του κύβου πρέπει να αθροίζουν στο 12) και τον κανόνα γωνίας (τρία κελιά σε κάθε γωνία του κύβου πρέπει επίσης να αθροίζουν στο 12). Αυτοί οι περιορισμοί δημιουργούν ένα μαθηματικά πιο σύνθετο πρόβλημα ικανοποίησης περιορισμών (Constraint Satisfaction Problem – CSP).

Η αρχιτεκτονική του συστήματος ακολουθεί το μοντέλο client-server με stateless API. Ο server είναι υπεύθυνος για τη δημιουργία γρίφων μοναδικής λύσης μέσω αλγορίθμου backtracking με ευρετική MRV (Minimum Remaining Values), την επικύρωση κινήσεων, το σύστημα υποδείξεων βασισμένο σε λογικό solver, καθώς και τη διαχείριση χρηστών μέσω JWT authentication και Google OAuth 2.0. Ο client παρέχει μια immersive τρισδιάστατη εμπειρία με δυναμικές κινήσεις κάμερας, animations μέσω GSAP, θεματικό σύστημα και διατήρηση προόδου μέσω localStorage.

Η εφαρμογή προσφέρει δύο επίπεδα δυσκολίας, Classic και BrainTerror, ημερήσιο γρίφο κοινό για όλους τους παίκτες, σύστημα βαθμολογίας και πίνακα κατάταξης, καθιστώντας την μια ολοκληρωμένη πλατφόρμα gaming.

Λέξεις-κλειδιά: 3D Sudoku, Constraint Satisfaction Problem, React, Three.js, ASP.NET Core, JWT Authentication, Backtracking, MRV Heuristic, WebGL

Πίνακας Περιεχομένων

Copyright ©.....	2
Acknowledgments.....	3
Περίληψη.....	4
Πίνακας Περιεχομένων.....	5
Πίνακας Εικόνων.....	9
Πίνακας Πινάκων.....	10
Πίνακας Διαγραμμάτων.....	11
1. Εισαγωγή.....	12
1.1 Περιγραφή του Προβλήματος.....	12
1.2 Στόχοι της Πτυχιακής.....	12
1.3 Μεθοδολογία Ανάπτυξης.....	12
1.4 Δομή της Πτυχιακής.....	13
2. Θεωρητικό Υπόβαθρο & Σχετικές Τεχνολογίες.....	13
2.1 Το Sudoku ως Πρόβλημα Ικανοποίησης Περιορισμών (CSP).....	13
2.1.1 Ορισμός CSP.....	13
2.1.2 Το CubeDoku ως Επέκταση CSP.....	14
2.1.3 Τοπολογία Κύβου.....	15
2.2 Το Αλγόριθμοι Επίλυσης.....	16
2.2.1 Backtracking με MRV Heuristic.....	16
2.2.2 Λογικές Τεχνικές Επίλυσης.....	16
2.2.3 Λογικές Τεχνικές Επίλυσης.....	18
2.3 Τεχνολογικό Stack.....	19
2.3.1 Backend: ASP.NET Core 8.....	19
2.3.2 Frontend: React 19 + Three.js.....	19
2.3.3 Βάση Δεδομένων: PostgreSQL (neon.com).....	19
2.3.4 Authentication: JWT + Google OAuth 2.0.....	19
2.3.5 Ασφάλεια.....	19
2.4 Σχετικές Εφαρμογές και Λύσεις.....	19
2.4.1 Υπάρχουσες Παραλλαγές Sudoku.....	19
2.4.2 Διαφοροποίηση CubeDoku.....	20
3. Επισκόπηση Εφαρμογής.....	20
3.1 Λειτουργικά Modules.....	20
3.1.1 Module Δημιουργίας Γρίφων (Puzzle Generation Module).....	20
3.1.2 Module Επικύρωσης Κινήσεων (Move Validation Module).....	20
3.1.3 Module Βοηθειών (Hint System Module).....	20

3.1.4 Module Αυθεντικοποίησης & Χρηστών (Auth & User Module).....	20
3.1.5 Module Βαθμολογίας & Κατάταξης (Scoring & Leaderboard Module)	21
3.2 Ρόλοι Χρηστών.....	22
3.2.1 Guest (Ανώνυμος Παίκτης)	22
3.2.2 Authenticated User (Εγγεγραμμένος Παίκτης).....	22
4. Αρχιτεκτονική Συστήματος.....	22
4.1 Αρχιτεκτονική Υψηλού Επιπέδου.....	22
4.2 Ροή Δεδομένων.....	23
4.2.1 Ροή Εκκίνησης Παιχνιδιού.....	23
4.2.2 Ροή Τοποθέτησης Αριθμού.....	24
4.2.3 Ροή Αποθήκευσης Αποτελέσματος.....	24
4.3 Frontend Αρχιτεκτονική.....	25
4.3.1 Component Hierarchy.....	25
4.3.2 Component Hierarchy.....	25
4.3.3 Custom Hooks	25
4.4 Backend Αρχιτεκτονική.....	25
4.4.1 Controller Layer.....	26
4.4.2 Core Layer.....	26
4.4.3 Middleware Pipeline.....	26
4.5 Υποδομή & Deployment.....	26
4.5.1 Development Environment.....	26
4.5.2 Production Deployment (Μελλοντικά σχέδια).....	27
5. Σχεδιασμός Βάσης Δεδομένων.....	27
5.1 Μοντέλο Δεδομένων.....	27
5.1.1 Οντότητα User.....	27
5.1.2 Οντότητα GameResult.....	28
5.1.3 Entity-Relationship Diagram.....	29
5.2 ApplicationDbContext.....	29
5.3 Migrations.....	30
5.4 Ασφάλεια Δεδομένων.....	30
5.4.1 Entity-Relationship Diagram.....	30
5.4.2 Anti-Enumeration.....	30
5.4.3 Server-side Value Clamping.....	30
6. Σχεδιασμός Λογισμικού & Δομή Κώδικα	31
6.1 Δομή Project	31
6.2 Design Patterns.....	31
6.2.1 Separation of Concerns (Core vs Controllers).....	31

6.2.2 DTO Pattern.....	31
6.2.3 Primary Constructor DI.....	32
6.2.4 Provider Pattern (React Context).....	32
6.3 Αλγοριθμική Ανάλυση.....	32
6.3.1 Αλγόριθμος Δημιουργίας Γρίφου.....	32
6.3.2 Thread Safety	33
7. Λειτουργικότητα & Workflows.....	34
7.1 Workflow Εγγραφής & Σύνδεσης.....	34
7.1.1 Email/Password Registration	34
7.1.2 Google OAuth Sign-in.....	35
7.2 Workflow Παιχνιδιού	38
7.2.1 Έναρξη Νέου Γρίφου.....	38
7.2.2 Προβολή οδηγιών (How to play)	39
7.2.3 Τοποθέτηση Αριθμού.....	40
7.2.4 Σημειώσεις (Pencil Mode).....	41
7.2.5 Αναίρεση (Undo).....	42
7.2.6 Υποδείξεις (Hints).....	42
7.2.7 Επανεκκίνηση Γρίφου (Start Over).....	43
7.3 Workflow Ολοκλήρωσης.....	43
7.4 Workflow Διαχείρισης & Ρυθμίσεων	44
7.4.1 Workflow Ρυθμίσεων (Settings)	44
7.4.2 Workflow Προβολής Στατιστικών (My Stats).....	46
7.5 Workflow Παιχνιδιού	46
7.5.1 Έναρξη Νέου Γρίφου.....	46
7.5.2 Συντονισμός Tabs (Tab Sync)	47
7.5.3 Themes (Dark/Light)	47
7.5.4 Animation & Ήχοι.....	47
7.6 Workflow Προβολής Κατάταξης (Leaderboard).....	48
8. Στρατηγική Ελέγχου (Testing & QA).....	49
8.1 Μεθοδολογία Ελέγχου.....	49
8.1.1 Manual Testing.....	49
8.1.2 Algorithmic Verification.....	49
8.1.3 Security Testing.....	49
8.2 Development-Only Tools	50
8.2.1 Solution Endpoint.....	50
8.2.2 Console Logging.....	50
8.3 Ασφάλεια – Defense in Depth	50

8.3.1 Rate Limiting Policies	50
8.3.2 Security Headers	50
8.3.3 Server-side Lock Check	51
8.4 Γνωστοί Περιορισμοί.....	51
9. Συμπεράσματα & Μελλοντικές Επεκτάσεις.....	51
9.1 Σύνοψη Επιτευγμάτων.....	51
9.2 Μελλοντικές Επεκτάσεις.....	52
9.2.1 Βραχυπρόθεσμες	52
9.2.2 Μεσοπρόθεσμες	52
9.2.3 Μεσοπρόθεσμες	52
9.3 Επίλογος.....	52
Βιβλιογραφία	53

Πίνακας Εικόνων

Εικόνα 1: Σελίδα καλωσορίσματος (Welcome Modal)	13
Εικόνα 2: Απεικόνιση 3 ακμών στον κύβο	14
Εικόνα 3: Απεικόνιση τρίπλετας στον κύβο	15
Εικόνα 4: Απεικόνιση 3 εδρών στον κύβο	15
Εικόνα 5: User Class	16
Εικόνα 6: Παράδειγμα Naked Single	17
Εικόνα 7: Παράδειγμα Hidden Single	17
Εικόνα 8: Παράδειγμα 12-Sum Deduction.....	18
Εικόνα 9: Παράδειγμα Naked Pairs.....	18
Εικόνα 10: Απεικόνιση score, δίπλα στο χρονόμετρο	22
Εικόνα 11: User Class.....	28
Εικόνα 12: ApplicationDbContext Class	29
Εικόνα 13: Δημιουργία αντικειμένου χρήστη, χρησιμοποιώντας adaptive hashing για τον κωδικό	30
Εικόνα 14: Δημιουργία αντικειμένου κατόπιν ολοκλήρωσης παιχνιδιού	30
Εικόνα 15: Δομή του Project: Client-side & Server Side	31
Εικόνα 16: Κλάση AuthController	32
Εικόνα 17: Χρήση React Provider σε AuthContext	32
Εικόνα 18: Χρήση Lock κατά την δημιουργία puzzle	34
Εικόνα 19: Authentication μέσω GoogleID με JWT	35
Εικόνα 20: Σελίδα καλωσορίσματος, χωρίς ο χρήστης να είναι συνδεδεμένος	35
Εικόνα 21: Σελίδα σύνδεσης χρήστη	36
Εικόνα 22: Σελίδα δημιουργίας λογαριασμού	36
Εικόνα 23: Σελίδα επιτυχής σύνδεσης χρήστη	37
Εικόνα 24: Σελίδα καλωσορίσματος, ενώ ο χρήστης είναι συνδεδεμένος	37
Εικόνα 25: Σελίδα επιβεβαίωσης αποσύνδεσης χρήστη.....	38
Εικόνα 26: Σελίδα σύνδεσης χρήστη με το Google OAuth popup.....	38
Εικόνα 27: Ο κύβος σε Dark Theme	39
Εικόνα 28: Παράδειγμα Error State (κόκκινα κελιά).....	39
Εικόνα 29: Κομμάτι του HowToPlayModal	40
Εικόνα 30: Number Selector & Actions Buttons	41
Εικόνα 31: Έλεγχος click εάν είναι για τοποθέτηση αριθμού, ή rotate κύβου	41
Εικόνα 32: Παραδείγματα Pencil Notes	42
Εικόνα 33: Hint popup	42
Εικόνα 34: Popup επιβεβαίωσης χρήστη για επανεκκίνηση γρίφου	43
Εικόνα 35: Completion Modal, με 1 παίκτη, ως guest	44
Εικόνα 36: Completion Modal, με 2 παίκτες, ως συνδεδεμένος χρήστης	44
Εικόνα 37: SettingsModal στο Game Tab	45
Εικόνα 38: SettingsModal στο Account Tab.....	45
Εικόνα 39: Verify Identity popup κατά την αλλαγή στοιχείων λογαριασμού.....	46
Εικόνα 40: My Stats Modal	46
Εικόνα 41: Χρήση του BroadcastChannel API	47
Εικόνα 42: Σύγκριση Dark Theme με Light Theme	47
Εικόνα 43: Πατημένο κελί στον κύβο.....	48
Εικόνα 44: Leaderboard Modal	49
Εικόνα 45: DEV-Solution function.....	50
Εικόνα 46: Security headers	51
Εικόνα 47: Security headers	51

Πίνακας Πινάκων

Πίνακας 1: Custom hooks και η χρήση τους	25
Πίνακας 2: Τα Controllers και οι ρόλοι τους	26
Πίνακας 3: Περιεχόμενα οντότητας User	27
Πίνακας 4: Περιεχόμενα οντότητας GameResult	29
Πίνακας 5: Policies και οι ιδιότητες τους.....	50

Πίνακας Διαγραμμάτων

Διάγραμμα 1: Απεικόνιση αλληλεπίδρασης modules	21
Διάγραμμα 2: Αρχιτεκτονική υψηλού επιπέδου	23
Διάγραμμα 3: Απεικόνιση ροής εκκίνησης παιχνιδιού	23
Διάγραμμα 4: Απεικόνιση ροής τοποθέτησης αριθμού	24
Διάγραμμα 5: Απεικόνιση ροής αποθήκευσης αποτελεσμάτων	24
Διάγραμμα 6: Ιεραρχία project	25
Διάγραμμα 7: Απεικόνιση του Middleware pipeline	26
Διάγραμμα 8: ER Diagram για την σχέση Users-GameResults.....	29
Διάγραμμα 9: FlowChart Αλγόριθμου Δημιουργίας Γρίφου	33
Διάγραμμα 10: Αναλυτικό FlowChart Αλγόριθμου Δημιουργίας Γρίφου	34

1. Εισαγωγή

1.1 Περιγραφή του Προβλήματος

Το Sudoku αποτελεί ένα από τα πιο δημοφιλή παζλ λογικής παγκοσμίως, με εκατομμύρια παίκτες να ασχολούνται καθημερινά με την επίλυσή του σε εφημερίδες, εφαρμογές κινητών και ιστοσελίδες. Παρά την τεράστια δημοτικότητα του, η βασική μορφή του Sudoku, ένα δισδιάστατο πλέγμα 9×9 , έχει παραμείνει σχεδόν αμετάβλητη εδώ και δεκαετίες. Οι περισσότερες παραλλαγές περιορίζονται σε αλλαγές μεγέθους πλέγματος ή πρόσθετους δισδιάστατους περιορισμούς, όπως είναι το Killer Sudoku και το Samurai Sudoku.

Η μετάβαση του Sudoku σε τρισδιάστατο χώρο αποτελεί μια ευκαιρία που συνδυάζει τη λογική πρόκληση με μια νέα χωρική διάσταση. Ένας κύβος με $6 \text{ έδρες} \times 9 \text{ κελιά ανά έδρα}$ (σύνολο 54 κελιά) εισάγει γεωμετρικούς περιορισμούς, ακμές και γωνίες, που δεν υπάρχουν στο κλασικό Sudoku. Αυτοί οι περιορισμοί μετασχηματίζουν το πρόβλημα σε ένα πολυδιάστατο CSP, δημιουργώντας ένα πιο σύνθετο και ενδιαφέρον παζλ.

Ταυτόχρονα, η απεικόνιση ενός τρισδιάστατου γρίφου σε οθόνη δισδιάστατης αναπαράστασης θέτει σημαντικές προκλήσεις σε επίπεδο User Experience (UX): ο παίκτης πρέπει να μπορεί να περιστρέφει τον κύβο, να αλληλεπιδρά με κελιά σε διαφορετικές έδρες, και να κατανοεί τη γεωμετρική σχέση μεταξύ γειτονικών κελιών χωρίς σύγχυση.

1.2 Στόχοι της Πτυχιακής

Οι βασικοί στόχοι της παρούσας εργασίας είναι:

1. **Σχεδιασμός ενός μαθηματικά ορθού συνόλου κανόνων** για τρισδιάστατο Sudoku σε κύβο, που να εξασφαλίζει ύπαρξη μοναδικής λύσης και λογική επιλυσιμότητα.
2. **Ανάπτυξη αλγορίθμων δημιουργίας γρίφων** (puzzle generation) με εγγύηση μοναδικής λύσης και ελεγχόμενη δυσκολία, χρησιμοποιώντας τεχνικές backtracking, MRV heuristic, και λογικής επίλυσης (Naked Singles, Hidden Singles, Naked Pairs, 12-Sum Deduction).
3. **Υλοποίηση immersive τρισδιάστατης διεπαφής** χρησιμοποιώντας WebGL μέσω Three.js/React Three Fiber, με ομαλή αλληλεπίδραση, animations και σύστημα θεμάτων.
4. **Σχεδιασμός ασφαλούς αρχιτεκτονικής client-server** με JWT authentication, Google OAuth 2.0, rate limiting, και defense-in-depth πρακτικές.
5. **Δημιουργία ολοκληρωμένης εμπειρίας gaming** με ημερήσιο γρίφο, σύστημα βαθμολογίας, πίνακα κατάταξης, σύστημα υποδείξεων, και διατήρηση προόδου παιχνιδιού.

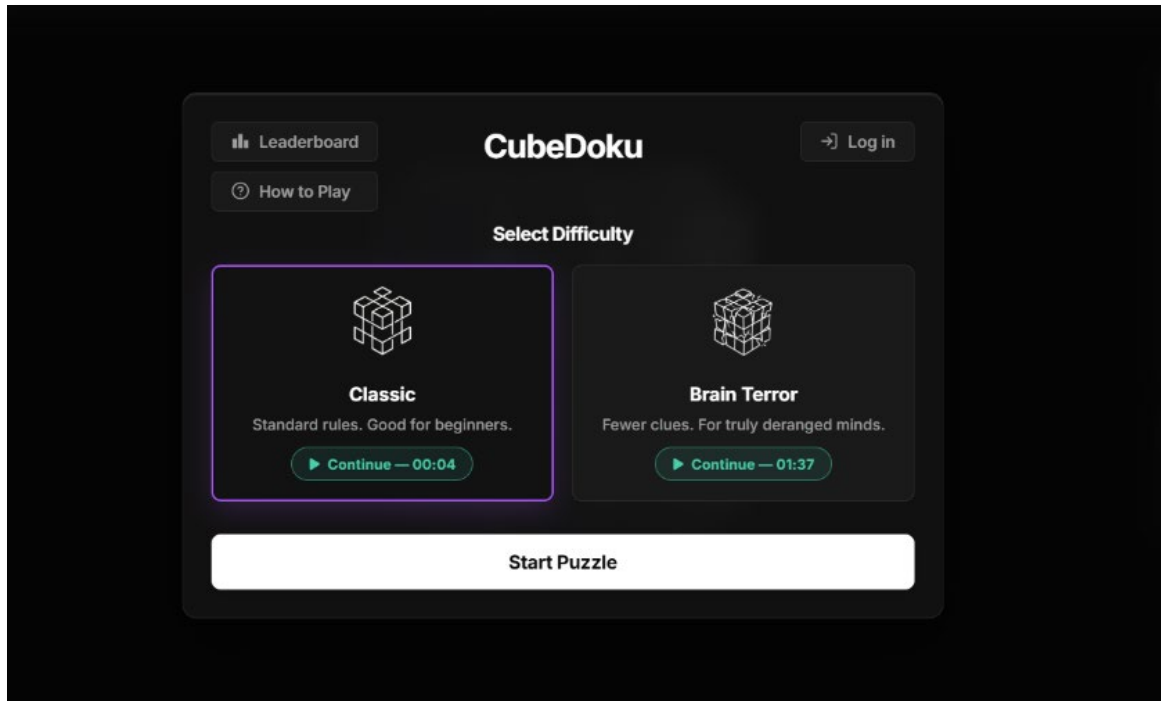
1.3 Μεθοδολογία Ανάπτυξης

Η ανάπτυξη ακολούθησε μια iterative, incremental προσέγγιση, χωρισμένη σε τέσσερις βασικές φάσεις:

- **Φάση 1 – Core Logic:** Σχεδιασμός και υλοποίηση των κανόνων του παζλ, του μοντέλου δεδομένων του κύβου (Cube, Cell, CellPosition), του αλγορίθμου backtracking (Solver), και του ελεγκτή εγκυρότητας (Checkers). Η συγκεκριμένη φάση θεωρείται και η πιο κρίσιμη, καθώς καθορίζει εάν ο γρίφος είναι μαθηματικά βιώσιμος.
- **Φάση 2 – Δημιουργία Γρίφων (Puzzle Generation):** Ανάπτυξη του PuzzleGenerator με στρατηγική αφαίρεσης κελιών (Center $\rightarrow$ Edge $\rightarrow$ Corner), του LogicalSolver για αξιολόγηση δυσκολίας, και του μηχανισμού εξασφάλισης μοναδικής λύσης μέσω CountSolutions.
- **Φάση 3 – Τρισδιάστατη Διεπαφή (3D Frontend):** Δημιουργία του 3D cube model στο Blender,

υλοποίηση του CubeViewer σε React Three Fiber, σχεδιασμός UI/UX με modals, animations, θέματα, και integration με το API.

- **Φάση 4 – Υποδομή & Ασφάλεια (Infrastructure & Security):** Υλοποίηση του backend API, βάσης δεδομένων, authentication, leaderboard, rate limiting, security headers, και προετοιμασία για production deployment.



Εικόνα 1: Σελίδα καλωσορίσματος (Welcome Modal)

1.4 Δομή της Πτυχιακής

Η εργασία είναι οργανωμένη ως εξής: Στο Κεφάλαιο 2 παρουσιάζεται το θεωρητικό υπόβαθρο και οι σχετικές τεχνολογίες. Στο Κεφάλαιο 3 γίνεται επισκόπηση της εφαρμογής. Στο Κεφάλαιο 4 αναλύεται η αρχιτεκτονική του συστήματος. Στο Κεφάλαιο 5 παρουσιάζεται ο σχεδιασμός της βάσης δεδομένων. Στο Κεφάλαιο 7 αναλύεται η δομή του κώδικα. Στο Κεφάλαιο 7 περιγράφονται οι ροές εργασίας. Στο Κεφάλαιο 8 παρουσιάζεται η στρατηγική ελέγχου. Τέλος, στο Κεφάλαιο 9 συνοψίζονται τα συμπεράσματα και οι μελλοντικές επεκτάσεις.

2. Θεωρητικό Υπόβαθρο & Σχετικές Τεχνολογίες

2.1 Το Sudoku ως Πρόβλημα Ικανοποίησης Περιορισμών (CSP)

2.1.1 Ορισμός CSP

Ένα Πρόβλημα Ικανοποίησης Περιορισμών (Constraint Satisfaction Problem – CSP) ορίζεται τυπικά ως ένα τριπλέτο $\langle X, D, C \rangle$ όπου:

- $X = \{x_1, x_2, \dots, x_n\}$ είναι ένα σύνολο μεταβλητών
- $D = \{D_1, D_2, \dots, D_n\}$ είναι τα αντίστοιχα πεδία τιμών
- $C = \{C_1, C_2, \dots, C_m\}$ είναι ένα σύνολο περιορισμών

Στο κλασικό Sudoku 9×9 , κάθε κελί είναι μια μεταβλητή, το πεδίο τιμών κάθε μεταβλητής είναι $\{1, \dots, 9\}$,

και οι περιορισμοί απαιτούν μοναδικότητα τιμών σε κάθε γραμμή, στήλη και τετράγωνο 3×3 .

2.1.2 Το CubeDoku ως Επέκταση CSP

Στο CubeDoku, το CSP επεκτείνεται σημαντικά:

- **Μεταβλητές:** 54 κελιά ($6 \text{ έδρες} \times 9 \text{ κελιά}$)
- **Πεδίο τιμών:** $\{1, 2, \dots, 9\}$ για κάθε κελί
- **Περιορισμοί:**
 - **Κανόνας Έδρας (Face Rule):** Δύο ή παραπάνω κελιά στην ίδια έδρα δεν μπορούν να έχουν την ίδια τιμή. (6 περιορισμοί $\times$ 36 ζεύγη ανά έδρα)
 - **Κανόνας Ακμής (Edge Rule):** Δύο κελιά που μοιράζονται φυσική ακμή κύβου πρέπει να αθροίζουν στο 12. (12 ζεύγη ακμών)
 - **Κανόνας Γωνίας (Corner Rule):** Τρία κελιά σε κάθε γωνία κύβου πρέπει να αθροίζουν στο 12. (8 τριπλέτες γωνιών)

Η τιμή 12 δεν επιλέχθηκε στην τύχη. Αντιπροσωπεύει το μοναδικό άθροισμα-στόχο που εξασφαλίζει ότι τόσο τα ζεύγη ακμών όσο και οι τριπλέτες γωνιών μπορούν πάντα να ικανοποιηθούν με τις τιμές 1-9. Για ζεύγη: $3+9=12$, $4+8=12$, $5+7=12$, $6+6=12$ (απαγορευμένο από τον κανόνα έδρας).

Για τριπλέτες: πολλοί συνδυασμοί (π.χ. $1+2+9$, $1+3+8$, $2+3+7$, κ.λπ.) ικανοποιούν τον περιορισμό.



Εικόνα 2: Απεικόνιση 3 ακμών στον κύβο



Εικόνα 3: Απεικόνιση τρίπλετας στον κύβο



Εικόνα 4: Απεικόνιση 3 εδρών στον κύβο

2.1.3 Τοπολογία Κύβου

Η τοπολογία του κύβου ορίζει ποια κελιά είναι «γειτονικά» μέσω κοινών ακμών ή γωνιών. Ο κύβος έχει:

- **6 έδρες:** Front, Back, Top, Bottom, Left, Right
- **12 ακμές:** κάθε ακμή συνδέει ένα κελί-ακμής μιας έδρας με ένα κελί-ακμής γειτονικής έδρας
- **8 γωνίες:** κάθε γωνία συνδέει τρία κελιά-γωνίας από τρεις γειτονικές έδρες

Κάθε κελί ανήκει σε μία από τρεις κατηγορίες ανάλογα με τη θέση του στην έδρα:

- **Κεντρικό (Center):** θέση (1,1) — συμμετέχει μόνο στον κανόνα έδρας
- **Κελί Ακμής (Edge):** θέσεις (0,1), (1,0), (1,2), (2,1) — συμμετέχει σε κανόνα έδρας + ένα ζεύγος ακμής

- **Κελί Γωνίας (Corner):** θέσεις (0,0), (0,2), (2,0), (2,2) — συμμετέχει σε κανόνα έδρας + μία τριπλέτα γωνίας

Αυτή η ιεραρχία περιορισμών (center < edge < corner) χρησιμοποιείται και από τον PuzzleGenerator για στρατηγική αφαίρεση κελιών.

2.2 Το Αλγόριθμοι Επίλυσης

2.2.1 Backtracking με MRV Heuristic

Ο βασικός αλγόριθμος επίλυσης χρησιμοποιεί αναδρομική οπισθοδρόμηση (**backtracking**) με την ευρετική Minimum Remaining Values (**MRV**). Η MRV, γνωστή και ως «fail-first heuristic», επιλέγει πρώτα το κελί με τις λιγότερες έγκυρες τιμές. Αυτό μειώνει δραστικά τον χώρο αναζήτησης, καθώς:

1. Αν ένα κελί δεν έχει καμία έγκυρη τιμή, η αποτυχία ανιχνεύεται αμέσως (fail-fast)
2. Κελιά με λίγες επιλογές τείνουν να δημιουργούν τις περισσότερες συγκρούσεις, επομένως η πρώιμη ανάθεση τους αποφεύγει μεγάλα backtrack βάθη

Η μέθοδος *GetBestCell* υλοποιεί αυτή την ευρετική:

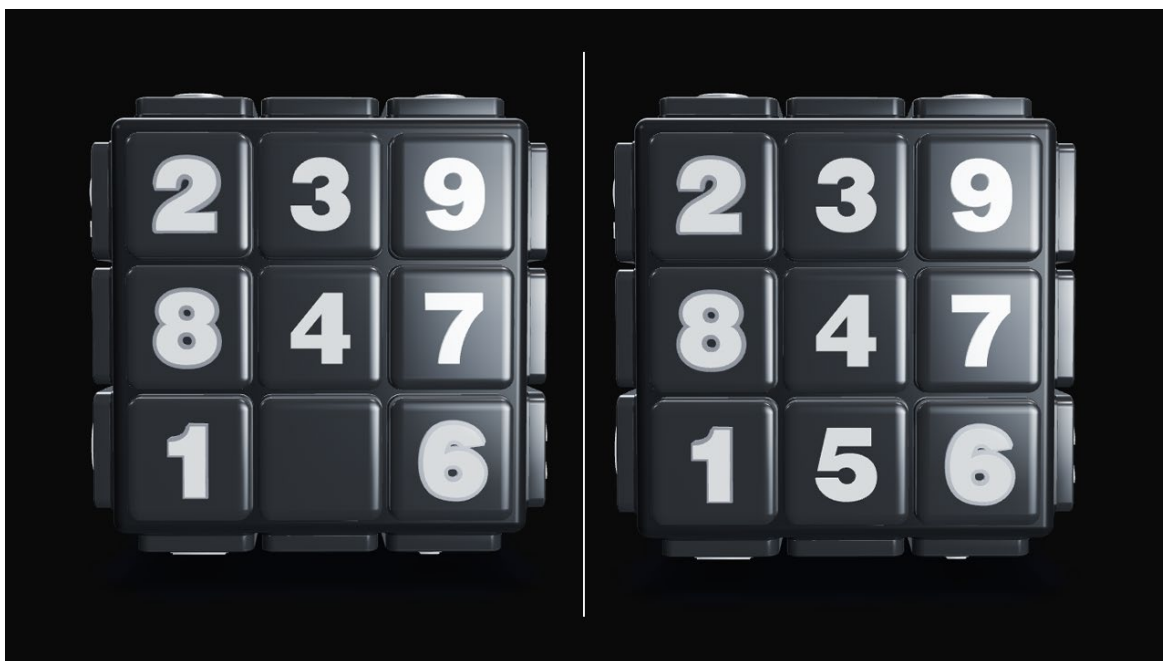
```
private Cell GetBestCell(Cube cube)
{
    Cell bestCell = null;
    int minOptions = 100;
    foreach (var face in Enum.GetValues<CubeFaces>())
    {
        for (int row = 0; row < 3; row++)
        {
            for (int column = 0; column < 3; column++)
            {
                var cell = cube.getCell(new CellPosition(face, row, column));
                if (cell.getNumber() != 0) continue;
                int validOptionsForThisCell = CountValidMoves(cube, cell);
                if (validOptionsForThisCell == 0) return cell; // fail-fast
                if (validOptionsForThisCell < minOptions)
                {
                    minOptions = validOptionsForThisCell;
                    bestCell = cell;
                }
            }
        }
    }
    return bestCell;
}
```

Εικόνα 5: User Class

2.2.2 Λογικές Τεχνικές Επίλυσης

Ο LogicalSolver υλοποιεί τεχνικές που μιμούνται την ανθρώπινη λογική, χωρίς backtracking:

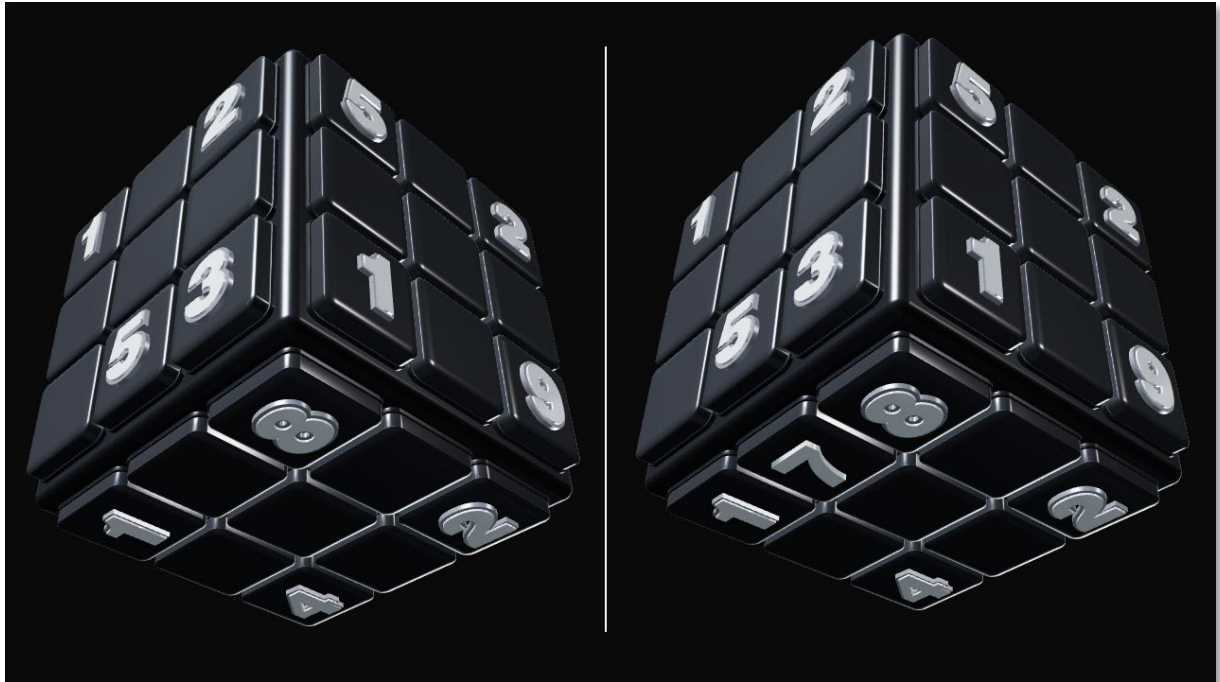
- **Naked Singles:** Ένα κελί με ακριβώς έναν υποψήφιο, η τιμή του είναι αναγκαστική. Αυτή είναι η πιο βασική τεχνική και χειρίζεται το μεγαλύτερο μέρος του γρίφου.
- **Hidden Singles:** Ένας αριθμός που μπορεί να τοποθετηθεί σε ένα μόνο κελί μιας έδρας. Ακόμα κι αν το κελί έχει πολλούς υποψήφιους, ο συγκεκριμένος αριθμός αναγκάζεται εκεί.
- **12-Sum Deduction:** Όταν ένα κελί σε ζεύγος ακμής ή τριπλέτα γωνίας είναι γνωστό, η τιμή του γειτονικού κελιού μπορεί να υπολογιστεί ως 12 – [γνωστές τιμές].
- **Naked Pairs:** Αν δύο κελιά σε μια έδρα μοιράζονται ακριβώς τους ίδιους δύο υποψήφιους, αυτές οι δύο τιμές μπορούν να αφαιρεθούν από τους υποψηφίους όλων των υπολοίπων κελιών της ίδιας έδρας.



Εικόνα 6: Παράδειγμα Naked Single



Εικόνα 7: Παράδειγμα Hidden Single



Εικόνα 8: Παράδειγμα 12-Sum Deduction



Εικόνα 9: Παράδειγμα Naked Pairs

2.2.3 Λογικές Τεχνικές Επίλυσης

Η δυσκολία ενός γρίφου καθορίζεται αυτόματα από τις τεχνικές που απαιτούνται για την επίλυσή του:

- **Classic:** Απαιτεί μόνο Naked Singles και Hidden Singles (~25 starting clues)
- **BrainTerror:** Απαιτεί επιπλέον Naked Pairs και 12-Sum Deduction (~ 15 starting clues)

2.3 Τεχνολογικό Stack

2.3.1 Backend: ASP.NET Core 8

Η επιλογή του ASP.NET Core 8 βασίστηκε σε πολλαπλούς παράγοντες:

- **Απόδοση:** Σταθερά κατατάσσεται μεταξύ των ταχύτερων web frameworks στα *TechEmpower* benchmarks, κρίσιμο για τις υπολογιστικά εντατικές λειτουργίες παραγωγής γρίφων
- **Type Safety:** Η στατική τυποποίηση της C# μειώνει runtime errors σε σύνθετα domain models
- **Entity Framework Core:** Παρέχει code-first μετανάστευσης βάσης δεδομένων και LINQ queries
- **Ενσωματωμένη ασφάλεια:** Ολοκληρωμένη υποστήριξη JWT, CORS, rate limiting, Data Annotations

2.3.2 Frontend: React 19 + Three.js

Το frontend stack αποτελείται από:

- **React 19:** Component-based αρχιτεκτονική με hooks (useState, useEffect, useRef, useMemo, useCallback) για διαχείριση κατάστασης. Σύγχρονο framework κατάλληλο για μοντέρνα σχεδίαση
- **React Three Fiber:** Declarative React wrapper γύρω από το Three.js, που επιτρέπει τη χρήση JSX για 3D scenes
- **Three.:** Βιβλιοθήκη WebGL για rendering τρισδιάστατων γραφικών
- **GSAP:** Βιβλιοθήκη animation για ομαλές μεταβάσεις χρωμάτων materials και press animations
- **Vite 7:** Σύγχρονο build tool με Hot Module Replacement (HMR) για γρήγορη ανάπτυξη
- **@react-three/drei:** Βιβλιοθήκη με utilities (OrbitControls, Clone, Center, ContactShadows, Environment) που απλοποιούν τη δημιουργία 3D σκηνών

2.3.3 Βάση Δεδομένων: PostgreSQL (neon.com)

Η PostgreSQL επιλέχθηκε για:

- Αξιοπιστία και ωριμότητα σε production περιβάλλοντα
- Εγγενή υποστήριξη DateOnly (σημαντικό για ημερήσιους γρίφους)
- Υποστήριξη UUID τύπου δεδομένων για τα GUIDs χρηστών
- Transient fault handling μέσω EnableRetryOnFailure του Npgsql

Επιλέχθηκε ο πάροχος neon.com καθώς παρέχει δωρεάν πλάνο με ευρωπαϊκούς servers, κατάλληλο για το συγκεκριμένο project.

2.3.4 Authentication: JWT + Google OAuth 2.0

Η αυθεντικοποίηση βασίζεται σε δύο μηχανισμούς:

- **JWT (JSON Web Tokens):** Υπογεγραμμένα tokens HS256 με 60λεπτη λήξη, που περιέχουν user ID, email, username
- **Google OAuth 2.0:** Δυνατότητα σύνδεσης μέσω λογαριασμού Google, με server-side επικύρωση μέσω *Google.Apis.Auth*

2.3.5 Ασφάλεια

Η ασφάλεια υλοποιείται σε πολλαπλά επίπεδα (defense in depth):

- **Rate Limiting:** Fixed-window πολιτικές ανά endpoint category
- **Security Headers:** CSP, X-Frame-Options, X-Content-Type-Options, HSTS
- **Input Validation:** DataAnnotations + server-side επικύρωση
- **Password Hashing:** BCrypt με αυτόματο salting

[ΕΙΣΑΓΩΓΗ ΕΙΚΟΝΑΣ/ΔΙΑΓΡΑΜΜΑΤΟΣ: Διάγραμμα του τεχνολογικού stack σε μορφή πυραμίδας — Database (PostgreSQL) → Backend (ASP.NET Core 8) → API Layer (REST/JSON) → Frontend (React + Three.js) → Browser (We Τεχνολογικό Stack

2.4 Σχετικές Εφαρμογές και Λύσεις

2.4.1 Υπάρχουσες Παραλλαγές Sudoku

Στην αγορά υπάρχουν αρκετές παραλλαγές Sudoku:

- **Wordle/NYT Games:** Ημερήσια παζλ με κοινό γρίφο για όλους, ένα μοντέλο που προτιμήθηκε και χρησιμοποιήθηκε στο CubeDoku
- **Sudoku.com:** Κλασικό 2D Sudoku με hint system και difficulty levels
- **3D Sudoku apps:** Ελάχιστες υλοποιήσεις υπάρχουν, κυρίως σε mobile και steam, χωρίς real-time 3D rendering

2.4.2 Διαφοροποίηση CubeDoku

Το CubeDoku διαφοροποιείται μέσω:

- **Πραγματικής 3D απεικόνισης** με WebGL αντί απλοποιημένων 2D projections
- **Πρωτότυπων γεωμετρικών κανόνων** (Άθροισμα 12 σε ακμές/γωνίες)
- **Server-authoritative validation** αντί client-only λογικής
- **Λογικού hint system** που εξηγεί την αιτιολογία κάθε υπόδειξης
- **Ημερήσιου κοινού γρίφου** με ανταγωνιστικό leaderboard

3. Επισκόπηση Εφαρμογής

3.1 Λειτουργικά Modules

Η εφαρμογή CubeDoku αποτελείται από πέντε βασικές λειτουργικές μονάδες:

3.1.1 Module Δημιουργίας Γρίφων (Puzzle Generation Module)

Αυτό το module είναι υπεύθυνο για τη δημιουργία ημερησίων γρίφων μοναδικής λύσης. Λειτουργεί αποκλειστικά στον server και περιλαμβάνει:

- **PuzzleGenerator:** Ο κεντρικός αλγόριθμος που συντονίζει τη διαδικασία: δημιουργεί έναν λυμένο κύβο, αφαιρεί στρατηγικά κελιά, ελέγχει τη μοναδικότητα και τη δυσκολία
- **Solver:** Backtracking solver με MRV ευρετική για ολοκληρωμένη επίλυση και μέτρηση λύσεων
- **LogicalSolver:** Solver βασισμένος σε λογικές τεχνικές. Χρησιμοποιείται για αξιολόγηση δυσκολίας και δημιουργία hint

Η δημιουργία γίνεται μία φορά ημερησίως ανά επίπεδο δυσκολίας. Ο γρίφος αποθηκεύεται σε static cache και εξυπηρετείται σε όλους τους παίκτες. Ο seed βασίζεται στην ημερομηνία (YYYYMMDD), εξασφαλίζοντας ότι κάθε ημέρα θα παράγεται ένα νέο, μοναδικό puzzle, ανά δυσκολία.

3.1.2 Module Επικύρωσης Κινήσεων (Move Validation Module)

Κάθε τοποθέτηση αριθμού ελέγχεται από την πλευρά του server:

- Ανακατασκευή ολόκληρου του κύβου από τα δεδομένα που στέλνει ο client (stateless σχεδίαση)
- Εκτέλεση IndividualChecker στο κελί-στόχο και σε όλα τα γειτονικά κελιά
- Επαναέλεγχος όλων των γεμισμένων κελιών για ανίχνευση cross-face σφαλμάτων
- Επιστροφή μόνο των κελιών που άλλαξαν κατάσταση
- Έλεγχος ολοκλήρωσης γρίφου (CheckIfSolved)

3.1.3 Module Βοηθειών (Hint System Module)

Το σύστημα βοηθειών λειτουργεί σε δύο φάσεις:

- **Φάση 1 – Ενημέρωση Λάθους:** Αν ο παίκτης έχει τοποθετήσει λανθασμένα έναν αριθμό, ο παίκτης ενημερώνεται ότι είναι λάθος. Ο server συγκρίνει τον τοποθετημένο αριθμό, με τον ορθό, και εάν δεν ταιριάζουν, ο παίκτης ενημερώνεται.
- **Φάση 1 – Λογική Βοήθεια (Hint):** Αν δεν υπάρχουν σφάλματα, εκτελείται ο LogicalSolver στην τρέχουσα κατάσταση και επιστρέφεται το πρώτο βήμα που αφορά κενό κελί, μαζί με αιτιολογία (π.χ. "Naked Single", "Edge 12-Sum: 12 - 5 = 7").

Οι λογικές βοήθειες είναι περιορισμένες: 5 για Classic, 3 για BrainTerror.

3.1.4 Module Αυθεντικοποίησης & Χρηστών (Auth & User Module)

Περιλαμβάνει τρεις τρόπους εγγραφής/σύνδεσης:

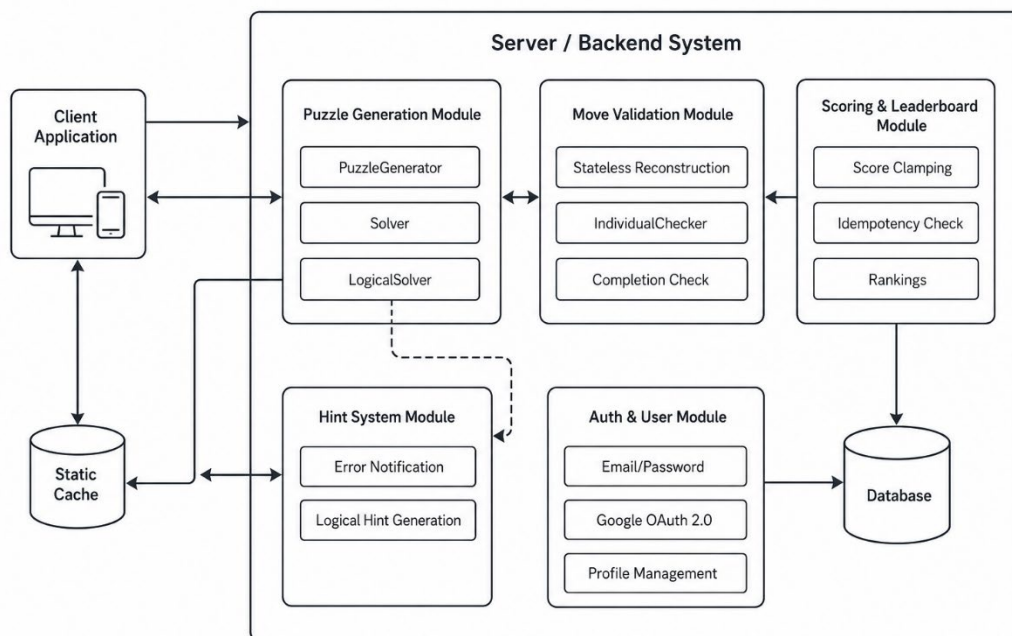
- Email + Password registration/login
- Google OAuth 2.0 sign-in
- Αυτόματη σύνδεση Google account με υπάρχοντα email account

Επιπλέον υπάρχει και διαχείριση προφίλ:

- Αλλαγή username (με έλεγχο μοναδικότητας)
- Αλλαγή password (με επικύρωση τρέχοντος κωδικού)
- Pre-validation password endpoint για βελτιωμένο UX

3.1.5 Module Βαθμολογίας & Κατάταξης (Scoring & Leaderboard Module)

- Υπολογισμός σκορ στον client βάσει χρόνου, λαθών, και βοηθειών
- Server-side clamping τιμών (Για αποτροπή cheating)
- Idempotency check: ένα αποτέλεσμα ανά χρήστη/ημέρα/δυσκολία
- "Nearby ranks" leaderboard: εμφάνιση ± 1 θέσεων γύρω από τον παίκτη
- Preview rank για guest παίκτες (χωρίς αποθήκευση σε περίπτωση επίλυσης)
- Ημερήσιο leaderboard με ταξινόμηση κατά σκορ $\rightarrow$ χρόνο $\rightarrow$ ημερομηνία υποβολής

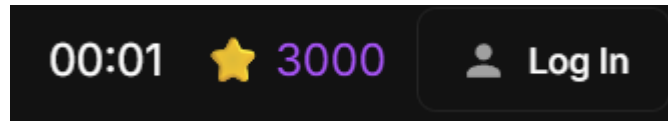


Διάγραμμα 1: Απεικόνιση αλληλεπίδρασης modules

Η βαθμολογία υπολογίζεται αποκλειστικά στον client και βασίζεται σε δύο συνιστώσες: τους **σταθερούς πόντους έδρας** και το **time bonus**. Κάθε φορά που ολοκληρώνεται μια έδρα (δηλαδή, και τα 9 κελιά της είναι γεμισμένα χωρίς σφάλματα), ο παίκτης λαμβάνει 500 πόντους βάσης. Στο ποσό αυτό προστίθεται ένα time bonus ίσο με $\max(0, 300 - \Delta t)$, όπου Δt είναι ο χρόνος σε δευτερόλεπτα που μεσολάβησε από την τελευταία ενέργεια του παίκτη έως τη στιγμή ολοκλήρωσης της έδρας. Συνεπώς, ο παίκτης ανταμείβεται για γρήγορη αλληλεπίδραση: μια ολοκλήρωση εντός 5 δευτερολέπτων αποφέρει $500 + 295 = 795$ πόντους, ενώ μια καθυστερημένη ολοκλήρωση πέρα των 5 λεπτών δίνει μόνο τους 500 πόντους βάσης.

Ο θεωρητικός μέγιστος βαθμός ανέρχεται σε $6 \times 800 = 4.800$ πόντους (6 έδρες $\times$ 500 + 300 μέγιστο bonus). Η βαθμολογία δεν τιμωρεί άμεσα τα λάθη ή τις υποδείξεις αριθμητικά, ωστόσο αυτά καθυστερούν τον παίκτη, μειώνοντας **έμμεσα** το time bonus. Κατά την αποθήκευση στον server, εφαρμόζεται clamping τιμών στο εύρος $[0, 10.000]$ μέσω `Math.Clamp`, ώστε αρνητικές τιμές ή υπερβολικά σκορ λόγω client-side manipulation να αποκλείονται.

Αξίζει να σημειωθεί ότι οι κινήσεις που προκύπτουν από τη λειτουργία hint δεν συνεισφέρουν σε πόντους: κατά τη χρήση hint, η παράμετρος `suppressScore` αποτρέπει την ενεργοποίηση του `scoring mechanism`. Αυτή η σχεδιαστική απόφαση εξασφαλίζει ότι ο παίκτης δεν μπορεί να εκμεταλλευτεί τις υποδείξεις για απόκτηση σκορ.



Εικόνα 10: Απεικόνιση score, δίπλα στο χρονόμετρο

3.2 Ρόλοι Χρηστών

3.2.1 Guest (Ανώνυμος Παίκτης)

Ο guest παίκτης μπορεί να:

- Παίξει οποιοδήποτε ημερήσιο γρίφο (Classic ή BrainTerror)
- Λάβει βοήθειες
- Δει preview του πίνακα κατάταξης (Leaderboard), χωρίς αποθήκευση αποτελεσμάτων
- Προβάλει τους κανόνες (How to Play)
- Αποθηκεύσει πρόοδο τοπικά (localStorage)

3.2.2 Authenticated User (Εγγεγραμμένος Παίκτης)

Πέρα από τις δυνατότητες guest, ο εγγεγραμμένος παίκτης μπορεί να:

- Αποθηκεύσει αποτελέσματα στη βάση δεδομένων
- Εμφανίζεται στον πίνακα κατάταξης (leaderboard)
- Βλέπει τα all-time προσωπικά στατιστικά του (Σύνολο παιχνιδιών, καλύτερο σκορ, κ.λπ.)
- Βλέπει τον καλύτερο χρόνο στον σημερινό γρύφο της κάθε δυσκολίας
- Διαχειρίζεται το προφίλ του (Αλλαγή του username & του κωδικού πρόσβασης)
-

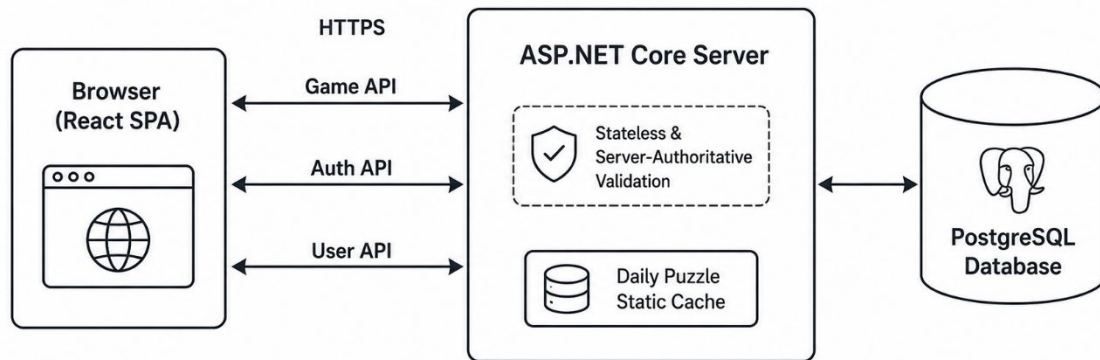
[ΕΙΣΑΓΩΓΗ ΕΙΚΟΝΑΣ/ΔΙΑΓΡΑΜΜΑΤΟΣ: Πίνακας σύγκρισης δυνατοτήτων Guest vs Authenticated User σε μορφή πίνακα με checkmarks]

4. Αρχιτεκτονική Συστήματος

4.1 Αρχιτεκτονική Υψηλού Επιπέδου

Το CubeDoku ακολουθεί **αρχιτεκτονική client-server** με τα εξής χαρακτηριστικά:

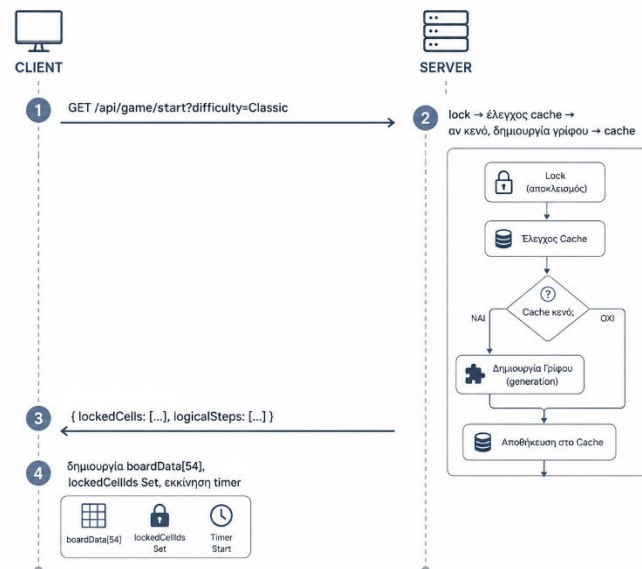
- **Stateless Server:** Ο server δεν διατηρεί game session state. Κάθε αίτημα περιέχει πλήρη κατάσταση πλακιδίων (54 τιμές). Αυτή η σχεδιαστική απόφαση απλοποιεί τη διαχείριση πολλαπλών tabs, αποφεύγει session management, και καθιστά τον server horizontally scalable.
- **Server-Authoritative Validation:** Παρόλο που ο client θα μπορούσε να εκτελεί επικύρωση τοπικά, η αυθεντική επικύρωση γίνεται αποκλειστικά στον server. Αυτό αποτρέπει client-side abuse.
- **Daily Puzzle Caching:** Η δημιουργία γρίφων είναι υπολογιστικά εντατική (seconds). Ο γρίφος δημιουργείται μία φορά ανά ημέρα ανά δυσκολία και αποθηκεύεται σε static cache, προστατευμένο με lock για thread safety.



Διάγραμμα 2: Αρχιτεκτονική υψηλού επιπέδου

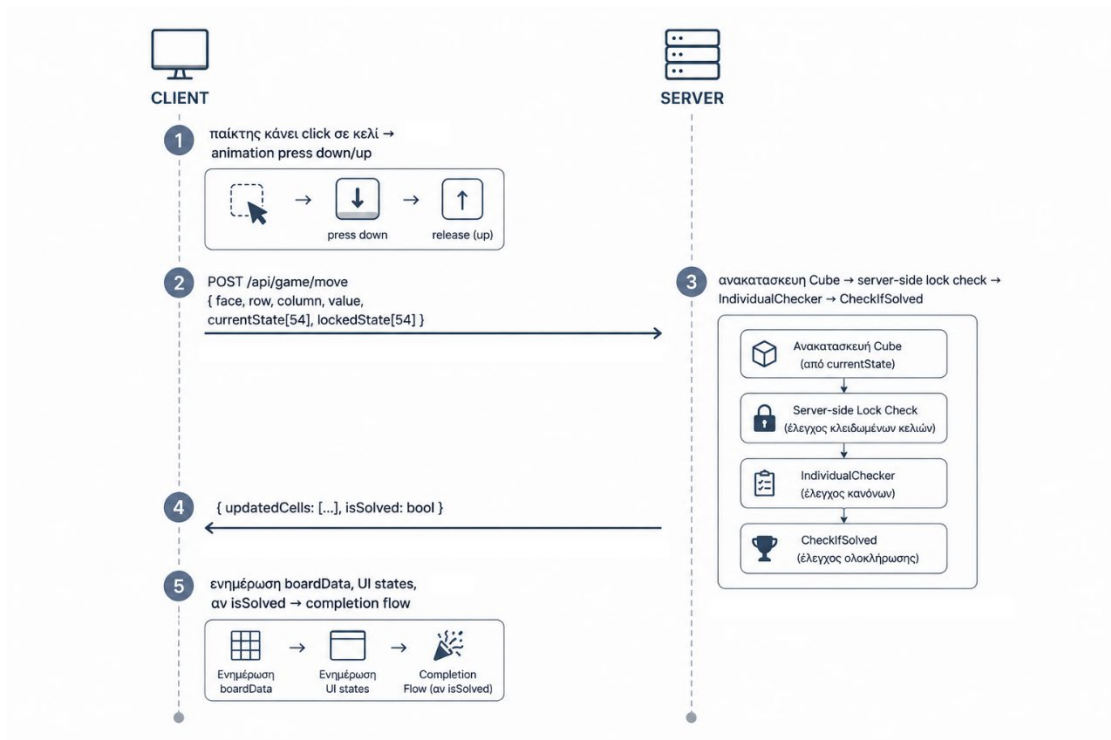
4.2 Ροή Δεδομένων

4.2.1 Ροή Εκκίνησης Παιχνιδιού



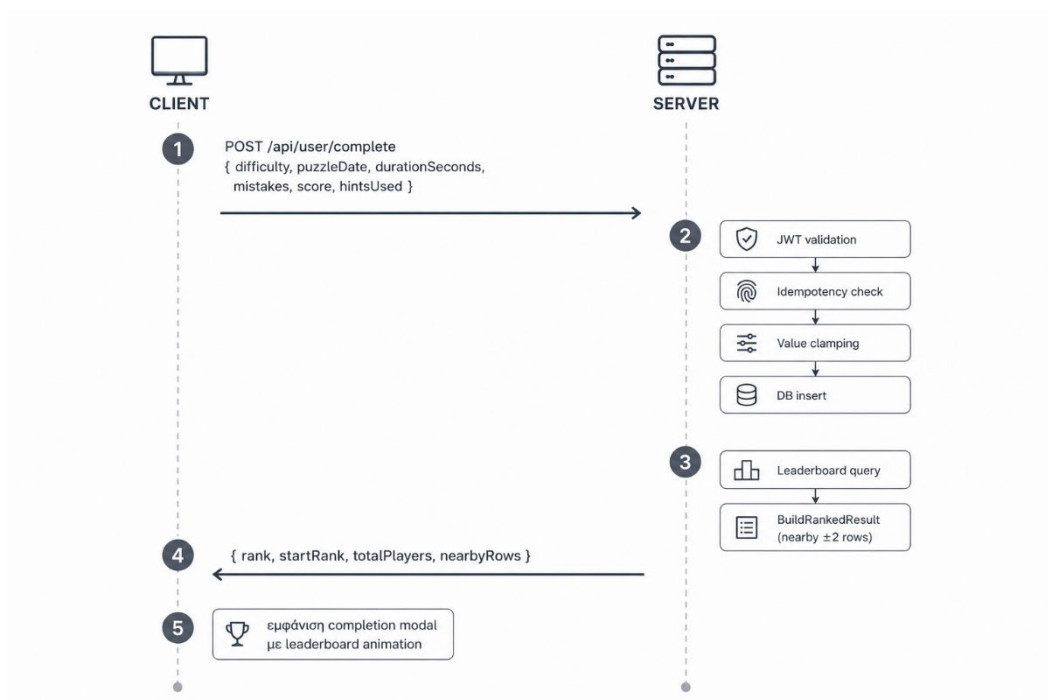
Διάγραμμα 3: Απεικόνιση ροής εκκίνησης παιχνιδιού

4.2.2 Ροή Τοποθέτησης Αριθμού



Διάγραμμα 4: Απεικόνιση ροής τοποθέτησης αριθμού

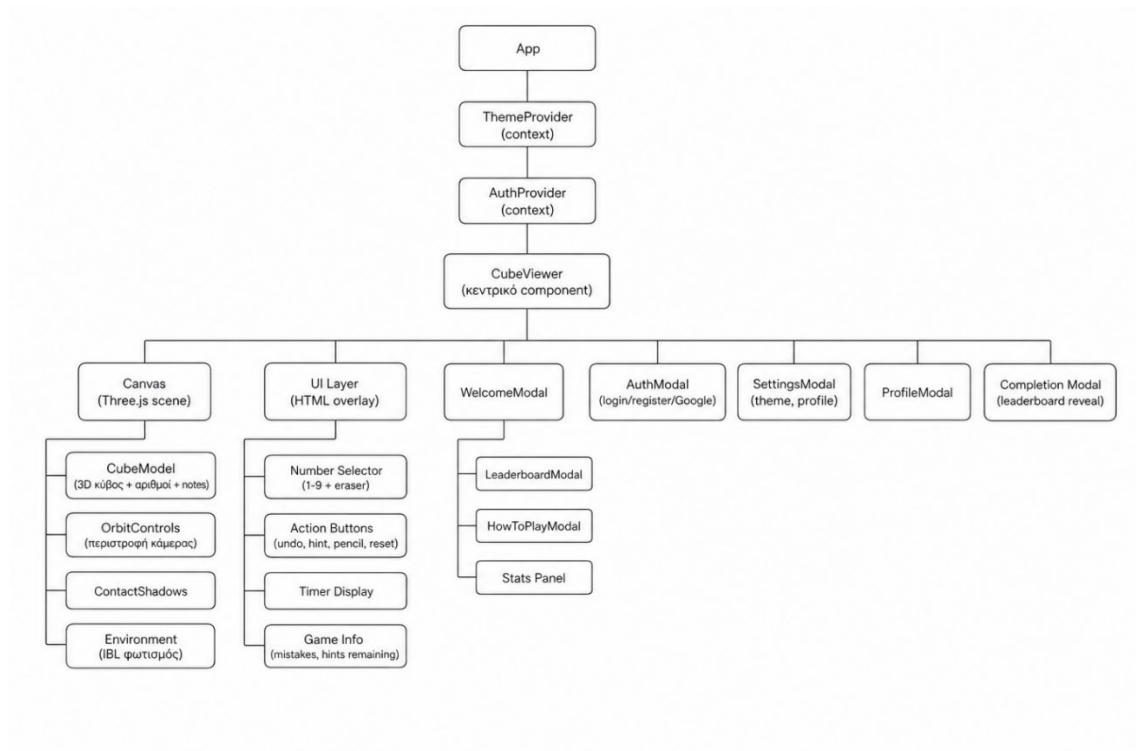
4.2.3 Ροή Αποθήκευσης Αποτελέσματος



Διάγραμμα 5: Απεικόνιση ροής αποθήκευσης αποτελεσμάτων

4.3 Frontend Αρχιτεκτονική

4.3.1 Component Hierarchy



Διάγραμμα 6: Ιεραρχία project

4.3.2 Component Hierarchy

Η εφαρμογή χρησιμοποιεί React Context API αντί εξωτερικής βιβλιοθήκης:

- **AuthContext:** Διαχείριση JWT token, user info, login/register/logout functions
- **ThemeContext:** Light/dark theme toggle, αποθήκευση σε localStorage, ανάγνωση system preference

Η κατάσταση παιχνιδιού (boardData, selectedNumber, mistakes, timer κ.λπ.) διαχειρίζεται τοπικά στο CubeViewer μέσω useState hooks, καθώς δεν χρειάζεται από άλλα components.

4.3.3 Custom Hooks

Hook	Χρήση
useModalTransition	Διαχείριση enter/exit animations, καθυστερεί unmount μέχρι να ολοκληρωθεί η CSS μετάβαση
useGamePersistence	Αποθήκευση/ανάκτηση in-progress game state σε localStorage
useTabSync	Συντονισμός sessions μεταξύ browser tabs μέσω BroadcastChannel API

Πίνακας 1: Custom hooks και η χρήση τους

4.4 Backend Αρχιτεκτονική

4.4.1 Controller Layer

Controller	Endpoints	Ρόλος
GameController	start, move, revert, hint, solution (dev)	Λογική παιχνιδιού
AuthController	register, login, google, profile/update, profile/validate-password	Αυθεντικοποίηση
UserController	complete, preview-rank, stats, today-best, leaderboard	Χρήστες & κατάταξη

Πίνακας 2: Τα Controllers και οι ρόλοι τους

4.4.2 Core Layer

Η μονάδα Core περιέχει τη domain logic, ανεξάρτητη από HTTP concerns:

- **Cube, Cell, CellPosition** — Domain models
- **CubeTopology** — Static lookup table ακμών και γωνιών
- **Checkers** — Validation engine (static + instance methods)
- **Solver** — Backtracking solver
- **LogicalSolver** — Ανθρώπινης λογικής solver
- **PuzzleGenerator** — Γεννήτρια πάζλ

4.4.3 Middleware Pipeline

Η σειρά του middleware pipeline είναι κρίσιμη στο ASP.NET Core:



Διάγραμμα 7: Απεικόνιση του Middleware pipeline

Αυτή η σειρά εξασφαλίζει ότι: τα security headers εφαρμόζονται πριν οτιδήποτε, το rate limiting γίνεται πριν την αυθεντικοποίηση (ώστε brute-force attacks να αποκλείονται πριν αξιολογηθεί το JWT), και το CORS πρέπει να προηγείται της αυθεντικοποίησης σύμφωνα με τις απαιτήσεις του ASP.NET Core.

4.5 Υποδομή & Deployment

4.5.1 Development Environment

- **IDE:** Visual Studio (ASP.NET + Vite SPA integration)

- **Dev Server:** Vite HMR στο port 5173, ASP.NET στο port 7055
- **SPA Proxy:** Αυτόματη εκκίνηση Vite dev server μέσω *SpaProxyLaunchCommand*

4.5.2 Production Deployment (Μελλοντικά σχέδια)

- **Backend:** Microsoft Azure (App Service)
- **Frontend:** Vercel (static hosting μετά από vite build)
- **Database:** Cloud-hosted PostgreSQL (Neon).

ΕΙΣΑΓΩΓΗ ΕΙΚΟΝΑΣ/ΔΙΑΓΡΑΜΜΑΤΟΣ: Deployment architecture diagram — Vercel (frontend CDN) → Azure App Service (ASP.NET backend) → PostgreSQL Database, με ξεχωριστά arrows για CORS, HTTPS, JWT flows]

5. Σχεδιασμός Βάσης Δεδομένων

5.1 Μοντέλο Δεδομένων

Η βάση δεδομένων αποτελείται από δύο οντότητες (entities) με σχέση 1-προς-πολλα:

5.1.1 Οντότητα User

Πεδίο	Τύπος	Περιγραφή
Id	Guid (PK)	Μοναδικό αναγνωριστικό, UUID αντί αυτο-αυξανόμενου int για αποτροπή enumeration
Username	string	Όνομα χρήστη που εμφανίζεται
Email	string	Μοναδικό e-mail, χρησιμοποιείτε για login
PasswordHash	string (μπορεί να είναι null)	BCrypt hash, null για Google-only accounts
GoogleID	string (μπορεί να είναι null)	Google Subject ID, null για email/password accounts
CreatedAt	DateTime	Ημερομηνία δημιουργίας λογαριασμού

Πίνακας 3: Περιεχόμενα οντότητας User

```

6 references
public class User
{
    // Guid because it's safer than auto-increment int
    // (can't guess other users' IDs by incrementing)
    6 references
    public Guid Id { get; set; } = Guid.NewGuid();

    12 references
    public string Username { get; set; } = string.Empty;

    11 references
    public string Email { get; set; } = string.Empty;

    // null means Google-only account (no password)
    10 references
    public string? PasswordHash { get; set; }

    // null means email/password account
    3 references
    public string? GoogleID { get; set; }

    // automatically set to now when the record is created
    0 references
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    // navigation property for EF - gives access to this user's game history
    0 references
    public ICollection<GameResult> GameResults { get; set; } = [];
}

```

Εικόνα 11: User Class

Η χρήση GUID ως primary key, αν και ελαφρώς λιγότερο αποδοτική σε indexing σε σχέση με ακέραιους, αποτρέπει επιθέσεις Insecure Direct Object Reference (**IDOR**), δηλαδή ένας κακόβουλος χρήστης δεν μπορεί να μαντέψει IDs άλλων χρηστών αυξάνοντας απλά έναν αριθμό.

5.1.2 Οντότητα GameResult

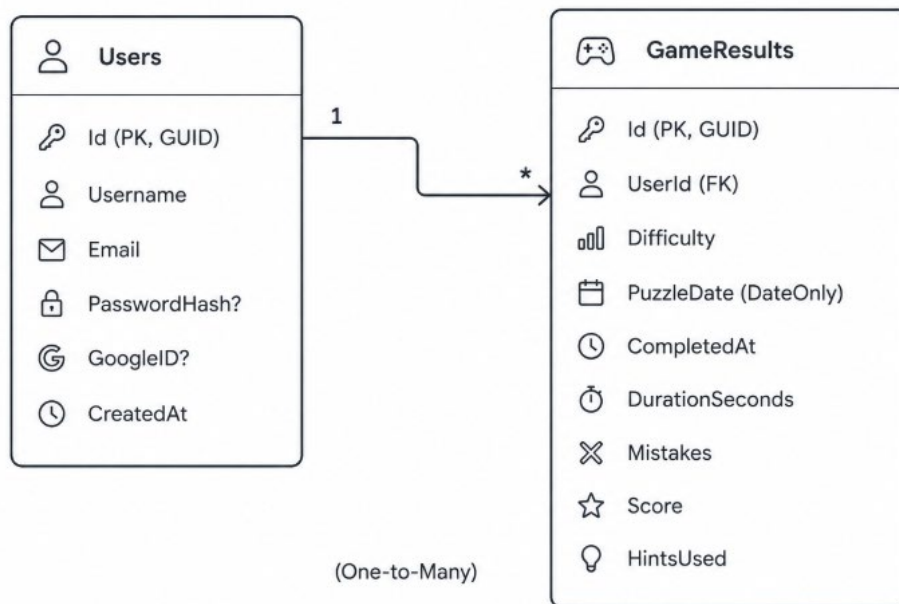
Πεδίο	Τύπος	Περιγραφή
Id	Guid (PK)	Μοναδικό αναγνωριστικό αποτελέσματος
UserId	Guid (FK)	Αναφορά στον χρήστη
Difficulty	string	“Classic” ή “BrainTerror”
PuzzleDate	DateOnly	Ημερομηνία γρύφου (χωρίς ώρα)
CompletedAt	DateTime	Στιγμή ολοκλήρωσης
DurationSeconds	int	Χρόνος ολοκλήρωσης, σε δευτερόλεπτα
Mistakes	int	Πλήθος σφαλμάτων

Score	int	Τελικό σκορ (0-10.000)
HintsUsed	int	Αριθμός βοηθειών που χρησιμοποιήθηκαν

Πίνακας 4: Περιεχόμενα οντότητας *GameResult*

Η χρήση *DateOnly* (αντί *DateTime*) για το *PuzzleDate* εξασφαλίζει ακριβή ομαδοποίηση ανά ημέρα χωρίς θέματα time zone, κάτι που καθιστάται κρίσιμο για τη λειτουργία ημερήσιου γρίφου.

5.1.3 Entity-Relationship Diagram

Διάγραμμα 8: ER Diagram για την σχέση *Users-GameResults*

5.2 ApplicationDbContext

Ο *DbContext* είναι σκόπιμα minimal:

```

8 references
public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
    : DbContext(options)
{
    // Users table - stores registered accounts
    8 references
    public DbSet<User> Users => Set<User>();

    // GameResults table - one row per completed puzzle per user
    6 references
    public DbSet<GameResult> GameResults => Set<GameResult>();
}
  
```

Εικόνα 12: *ApplicationDbContext* Class

Η εφαρμογή βασίζεται σε **EF Core conventions** χωρίς Fluent API configuration. Ο unique constraint (UserId + Difficulty + PuzzleDate) δεν υλοποιείται σε database level αλλά εφαρμόζεται μέσω application-level check στον UserController.

5.3 Migrations

Τρεις migrations τεκμηριώνουν την εξέλιξη του schema:

- **InitialCreate** (07/03/2026): Δημιουργία πινάκων Users και GameResults
- **RenameUserIdColumn** (07/03/2026): Μετονομασία column για συνέπεια naming
- **AddHintsUsedToGameResult** (28/04/2026): Προσθήκη πεδίου HintsUsed — added αργότερα όταν υλοποιήθηκε η λειτουργία counting hints

5.4 Ασφάλεια Δεδομένων

5.4.1 Entity-Relationship Diagram

Οι κωδικοί αποθηκεύονται ως BCrypt hashes με αυτόματο salting:

```
var user = new User
{
    Username    = req.Username,
    Email       = req.Email,
    PasswordHash = BCrypt.Net.BCrypt.HashPassword(req.Password)
};
```

Εικόνα 13: Δημιουργία αντικειμένου χρήστη, χρησιμοποιώντας adaptive hashing για τον κωδικό

Ο BCrypt χρησιμοποιεί adaptive hashing, ο cost factor αυξάνει αυτόματα τον χρόνο hashing, καθιστώντας brute-force attacks ακριβές.

5.4.2 Anti-Enumeration

Τα μηνύματα σφάλματος κατά την αυθεντικοποίηση είναι επίτηδες αόριστα:

- **Registration failure:** "Registration failed. Please check your details." (δεν αποκαλύπτει αν το email υπάρχει)
- **Login failure:** "Invalid e-mail or password." (ίδιο μήνυμα ανεξάρτητα αν είναι λάθος το e-mail ή ο κωδικός)

5.4.3 Server-side Value Clamping

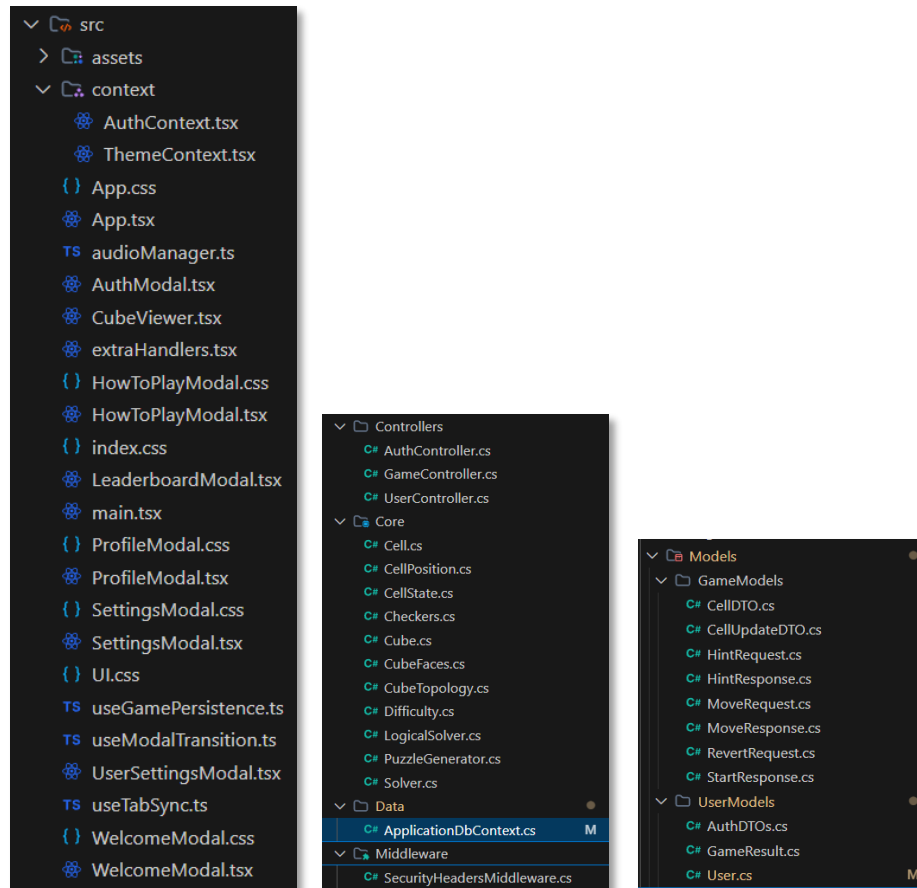
```
// clamp values to sane ranges in case the client sends something unreasonable
// (basic anti-cheat: doesn't stop determined attackers but filters obvious tampering)
var result = new GameResult
{
    UserId        = userId,
    Difficulty     = req.Difficulty,
    PuzzleDate    = req.PuzzleDate,
    DurationSeconds = Math.Max(0, req.DurationSeconds),
    Mistakes      = Math.Max(0, req.Mistakes),
    Score         = Math.Clamp(req.Score, 0, 10_000),
    HintsUsed     = Math.Max(0, req.HintsUsed)
};
```

Εικόνα 14: Δημιουργία αντικειμένου κατόπιν ολοκλήρωσης παιχνιδιού

Αυτό αποτρέπει αρνητικές τιμές ή υπερβολικά σκορ.

6. Σχεδιασμός Λογισμικού & Δομή Κώδικα

6.1 Δομή Project



Εικόνα 15: Δομή του Project: Client-side & Server Side

6.2 Design Patterns

6.2.1 Separation of Concerns (Core vs Controllers)

Το τμήμα Core δεν έχει καμία εξάρτηση από ASP.NET, HTTP, ή βάση δεδομένων. Αυτός ο διαχωρισμός σημαίνει ότι:

- Η λογική παιχνιδιού μπορεί να δοκιμαστεί ανεξάρτητα (unit tests χωρίς web server)
- Ο solver θα μπορούσε να επαναχρησιμοποιηθεί σε mobile app χωρίς περιορισμούς
- Αλλαγές στο API δεν επηρεάζουν τους αλγόριθμους

6.2.2 DTO Pattern

Τα Data Transfer Objects (**DTOs**) διαχωρίζουν τα εσωτερικά domain models από τα API contracts:

- *CellDTO*: απλοποιημένη αναπαράσταση κελιού (face, row, column, value)
- *CellUpdateDTO*: επεκτείνει με state το κελί (Default/Error/Completed)
- *MoveRequest/MoveResponse*: πλήρη request/response contracts
- *AuthDTOs* (records): immutable request objects με DataAnnotations

6.2.3 Primary Constructor DI

Οι controllers χρησιμοποιούν C# primary constructors για dependency injection:

```
public class AuthController(ApplicationDbContext db, IConfiguration config) : ControllerBase
{
```

Εικόνα 16: Κλάση AuthController

Αυτό μειώνει τον boilerplate κώδικα σε σχέση με παραδοσιακούς constructors με private fields.

6.2.4 Provider Pattern (React Context

Το AuthContext και ThemeContext ακολουθούν το React Provider pattern:

```
export const AuthProvider = ({ children }: { children: ReactNode }) => {
  // initialize from localStorage so the user stays logged in across refreshes
  const [token, setToken] = useState<string | null>(localStorage.getItem('token'));
  const [user, setUser] = useState<AuthUser | null>(null);

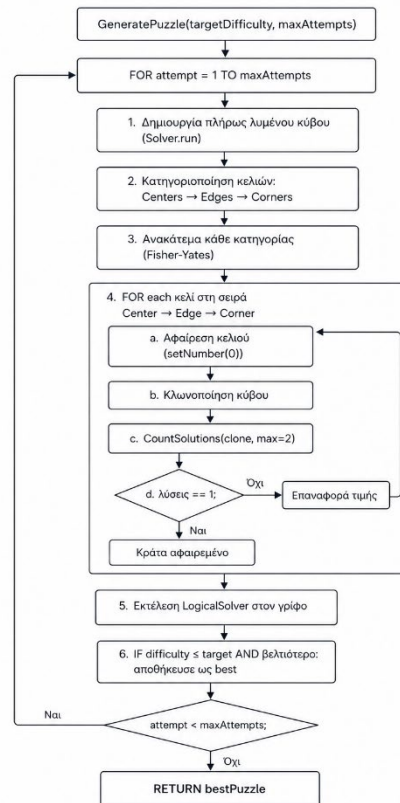
  return (
    <AuthContext.Provider value={{ user, token, login, register, loginWithGoogle, updateAuthToken: saveAuth, logout, isLoggedIn: !!user }}>
      {children}
    </AuthContext.Provider>
  );
};
```

Εικόνα 17: Χρήση React Provider σε AuthContext

Αυτό αποφεύγει prop drilling, οποιοδήποτε component μπορεί να χρησιμοποιήσει useAuth().

6.3 Αλγοριθμική Ανάλυση

6.3.1 Αλγόριθμος Δημιουργίας Γρίφου



Διάγραμμα 9: FlowChart Αλγόριθμου Δημιουργίας Γρίφου

Πολυπλοκότητα: Η δημιουργία είναι $O(A \times N \times S)$ όπου:

- A = attempts,
- N = κελιά προς αφαίρεση,
- S = κόστος CountSolutions.

Στην πράξη, ένας γρίφος δημιουργείται σε 2-5 δευτερόλεπτα.

6.3.2 Thread Safety

Η δημιουργία γρίφων χρησιμοποιεί **lock** για thread safety:

```

lock (_lock)
{
    // reset cache if the day has changed
    if (_cachedPuzzleDay != todaySeed)
    {
        _classicPuzzle = null;
        _brainterrorPuzzle = null;
        _classicSolution = null;
        _brainterrorSolution = null;
        _cachedPuzzleDay = todaySeed;
    }

    // generate Classic puzzle if not cached yet
    if (difficulty == "Classic" && _classicPuzzle == null)
    {
        var generator = new PuzzleGenerator(todaySeed);
        var result = generator.GeneratePuzzle(Difficulty.Classic);
        _classicPuzzle = ConvertToStartResponse(result.Puzzle, "Classic", result.Steps);
        _classicSolution = SolvePuzzleToFull(result.Puzzle, todaySeed);
    }

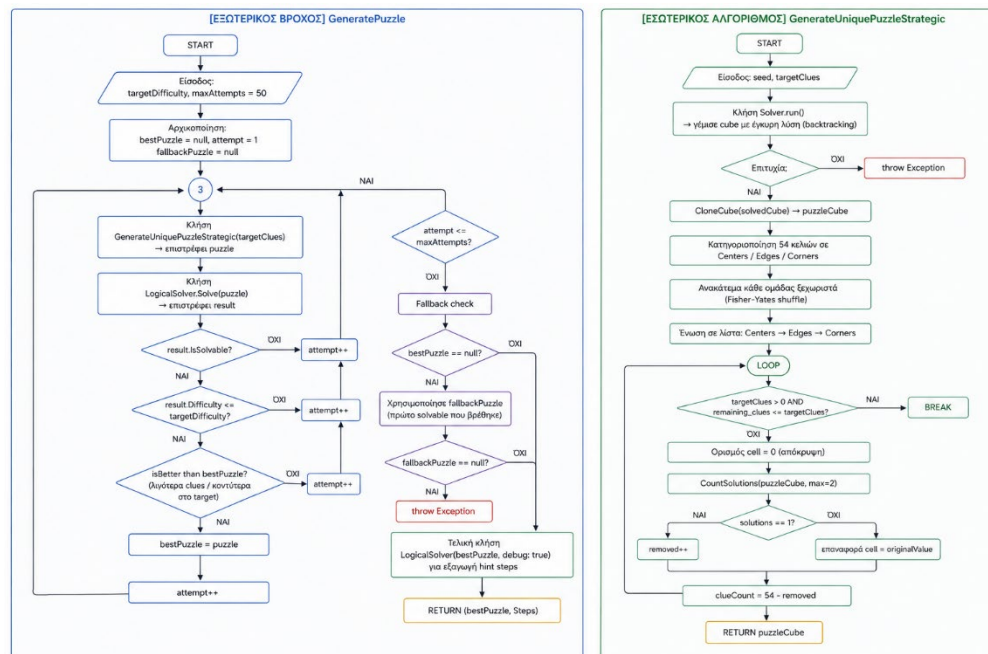
    // generate BrainTerror puzzle if not cached yet
    if (difficulty == "BrainTerror" && _brainterrorPuzzle == null)
    {
        var generator = new PuzzleGenerator(todaySeed);
        var result = generator.GeneratePuzzle(Difficulty.BrainTerror);
        _brainterrorPuzzle = ConvertToStartResponse(result.Puzzle, "BrainTerror", result.Steps);
        _brainterrorSolution = SolvePuzzleToFull(result.Puzzle, todaySeed);
    }

    var response = difficulty == "Classic" ? _classicPuzzle : _brainterrorPuzzle;
    return Ok(response);
}

```

Εικόνα 18: Χρήση Lock κατά την δημιουργία puzzle

Αυτό εξασφαλίζει ότι ακόμα κι αν πολλαπλά requests φτάσουν ταυτόχρονα στον server, ο γρίφος θα δημιουργηθεί μόνο μία φορά.



Διάγραμμα 10: Αναλυτικό FlowChart Αλγόριθμου Δημιουργίας Γρίφου

7. Λειτουργικότητα & Workflows

7.1 Workflow Εγγραφής & Σύνδεσης

7.1.1 Email/Password Registration

1. Ο χρήστης ανοίγει την εφαρμογή, βλέπει WelcomeModal, πατάει "Log in"

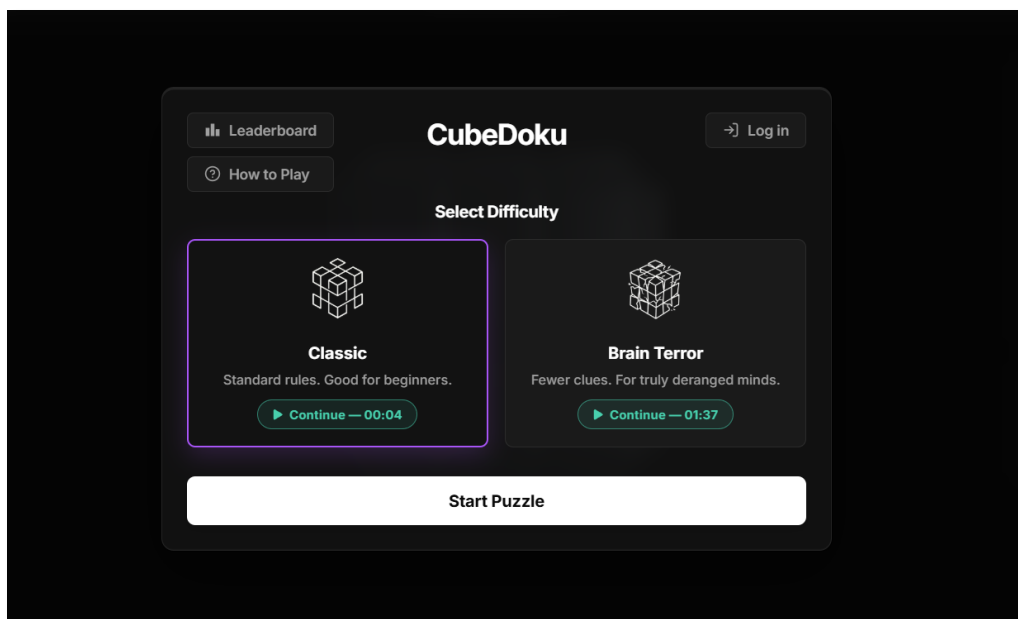
2. Εμφανίζεται το AuthModal με επιλογές Login / Register
3. Ο χρήστης εισάγει Username, Email, Password (min 8 χαρακτήρες)
4. Ο Client κάνει *POST /api/auth/register* με DataAnnotations validation
5. Ο Server ελέγχει εάν υπάρχει email (anti-enumeration generic error), έπειτα BCrypt hash και DB insert
6. Ο Server σε Client: JWT token + user info
7. Ο Client: *saveAuth(token)*, κατόπιν localStorage + state update, και τέλος redirect σε WelcomeModal

7.1.2 Google OAuth Sign-in

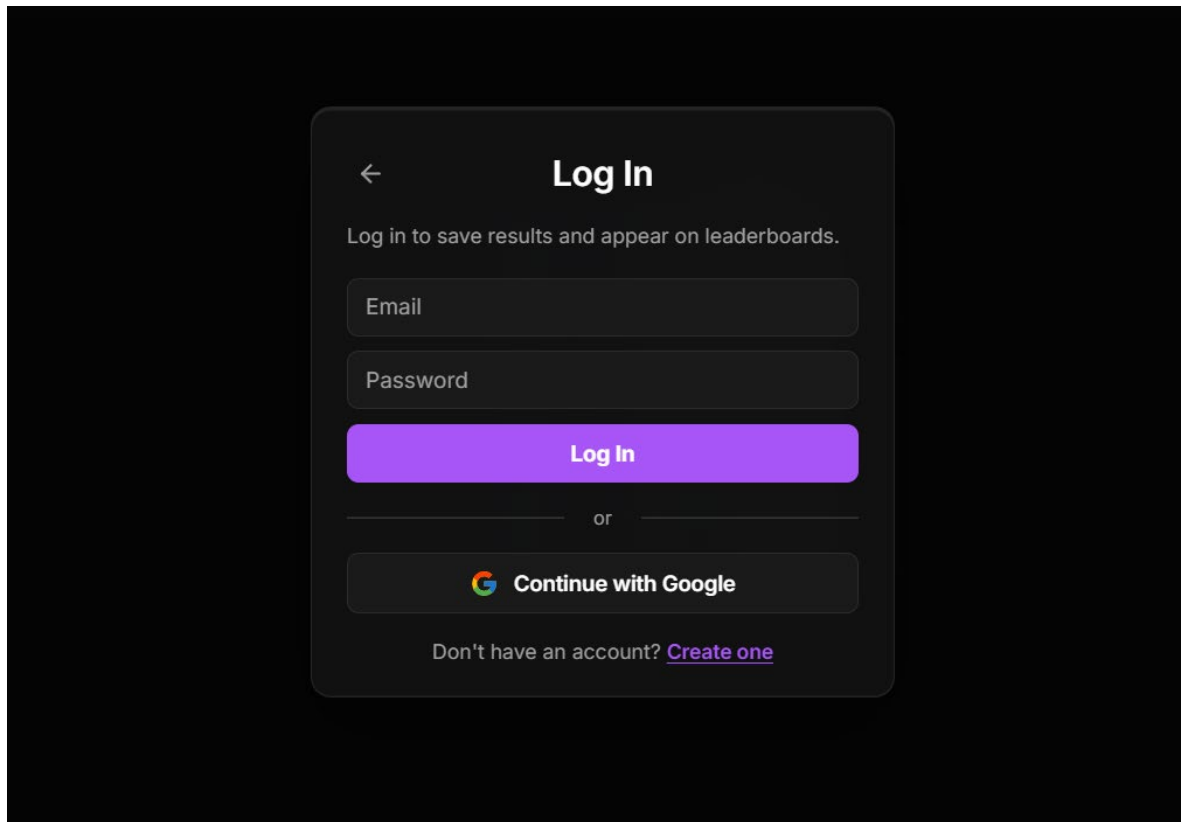
1. Ο χρήστης πατά "Sign in with Google" στο AuthModal
2. Εμφανίζεται Google OAuth popup, ο χρήστης αποδέχεται, Google ID Token
3. Ο Client κάνει *POST /api/auth/google* { idToken }
4. Ο Server: *GoogleJsonWebSignature.ValidateAsync* με audience check
5. Ο Server βρίσκει χρήστη με email, και δημιουργεί αν δεν υπάρχει / συνδέει GoogleID αν υπάρχει
6. Ο Server σε Client: JWT + user info

```
GoogleJsonWebSignature.Payload? payload;
try
{
    payload = await GoogleJsonWebSignature.ValidateAsync(
        req.IdToken,
        new GoogleJsonWebSignature.ValidationSettings
        {
            Audience = [clientId] // reject tokens that weren't issued for this app specifically
        });
}
```

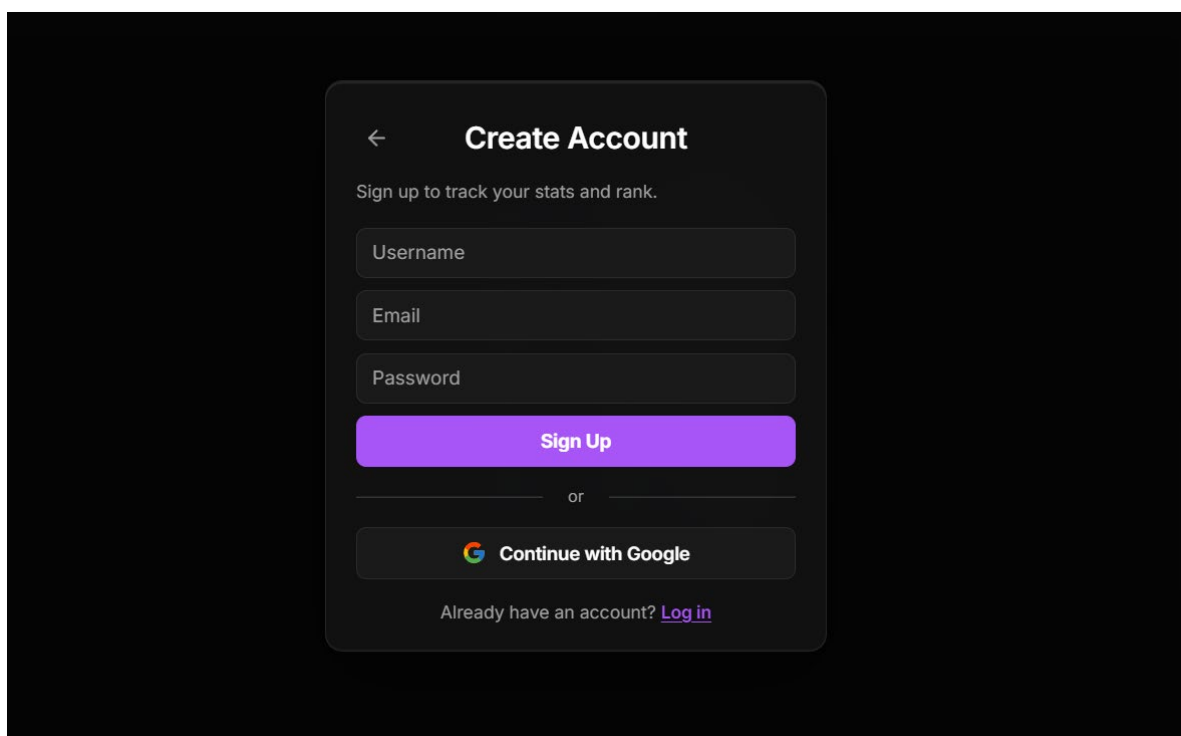
Εικόνα 19: Authentication μέσω GoogleID με JWT



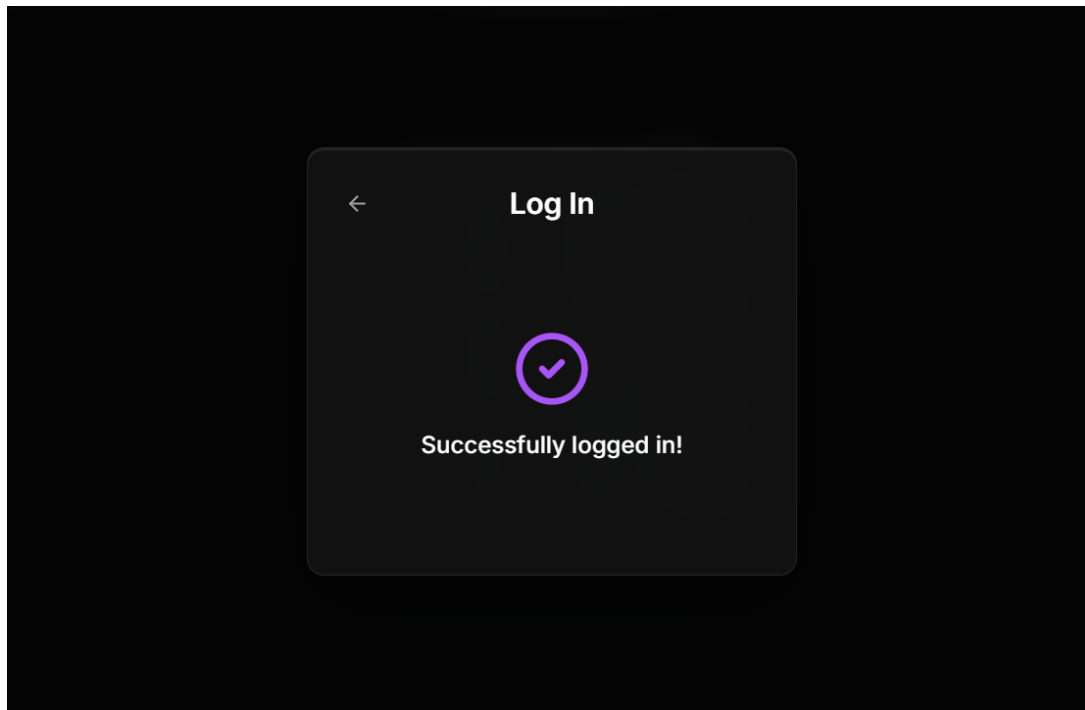
Εικόνα 20: Σελίδα καλωσορίσματος, χωρίς ο χρήστης να είναι συνδεδεμένος



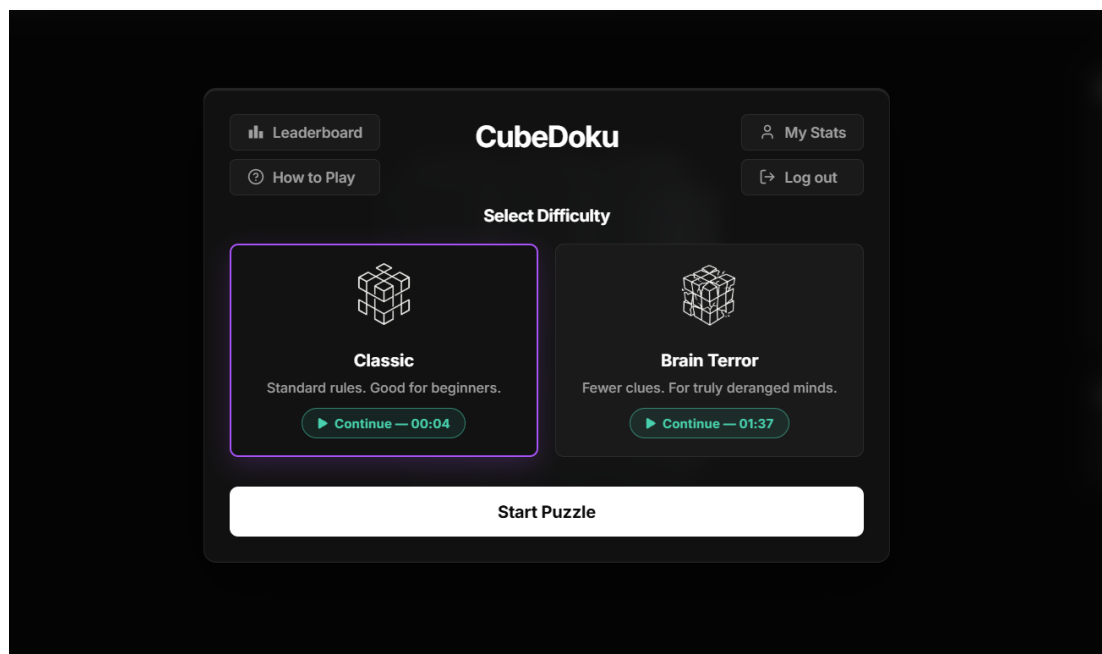
Εικόνα 21: Σελίδα σύνδεσης χρήστη



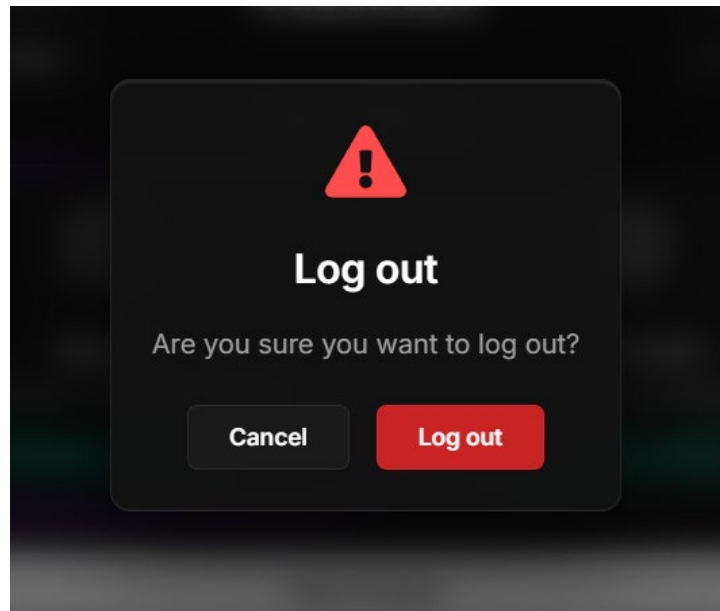
Εικόνα 22: Σελίδα δημιουργίας λογαριασμού



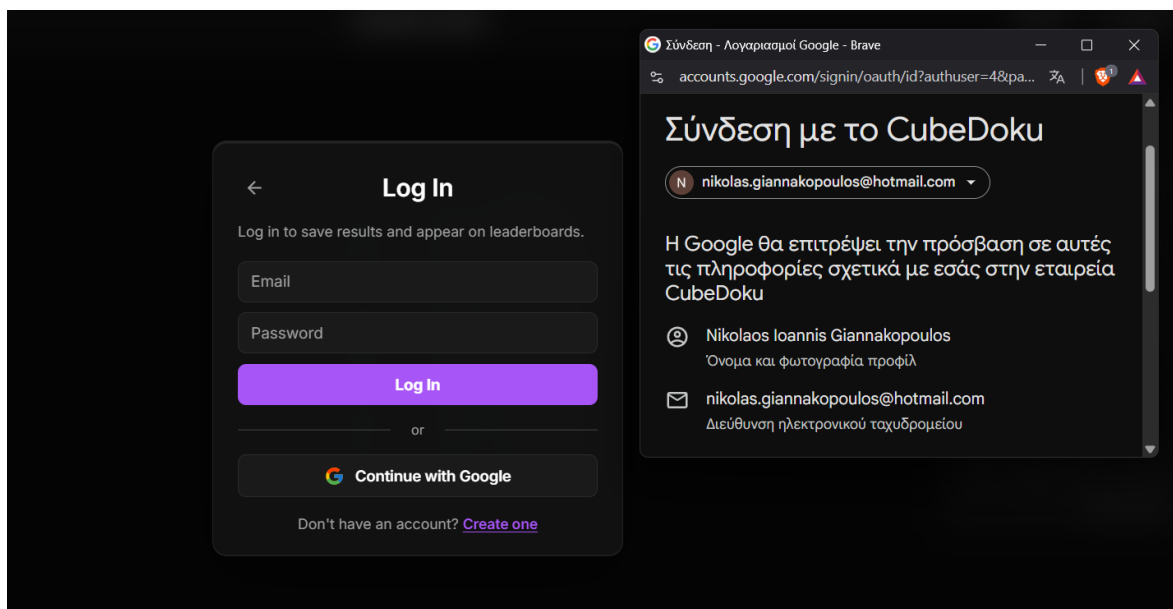
Εικόνα 23: Σελίδα επιτυχής σύνδεσης χρήστη



Εικόνα 24: Σελίδα καλωσορίσματος, ενώ ο χρήστης είναι συνδεδεμένος



Εικόνα 25: Σελίδα επιβεβαίωσης αποσύνδεσης χρήστη



Εικόνα 26: Σελίδα σύνδεσης χρήστη με το Google OAuth popup

7.2 Workflow Παιχνιδιού

7.2.1 Έναρξη Νέου Γρίφου

1. Στο WelcomeModal ο παίκτης επιλέγει δυσκολία (Classic ή BrainTerror)
2. Όταν γίνεται κλικ "Start Puzzle", ο Client στέλνει `GET /api/game/start?difficulty=Classic`
3. Ο Server κάνει lock, γίνεται η cache check, εάν χρειάζεται, γίνεται generation, στέλνει response
4. OClient δημιουργεί boardData[54], lockedCellIds Set
5. Το Canvas κάνει render 3D κύβου και τοποθετεί αριθμούς σε locked κελιά
6. Εκκίνηση Timer και εμφάνιση Number Selector / Tool Selector (1-9 + Eraser, Undo, Hint, Pencil)



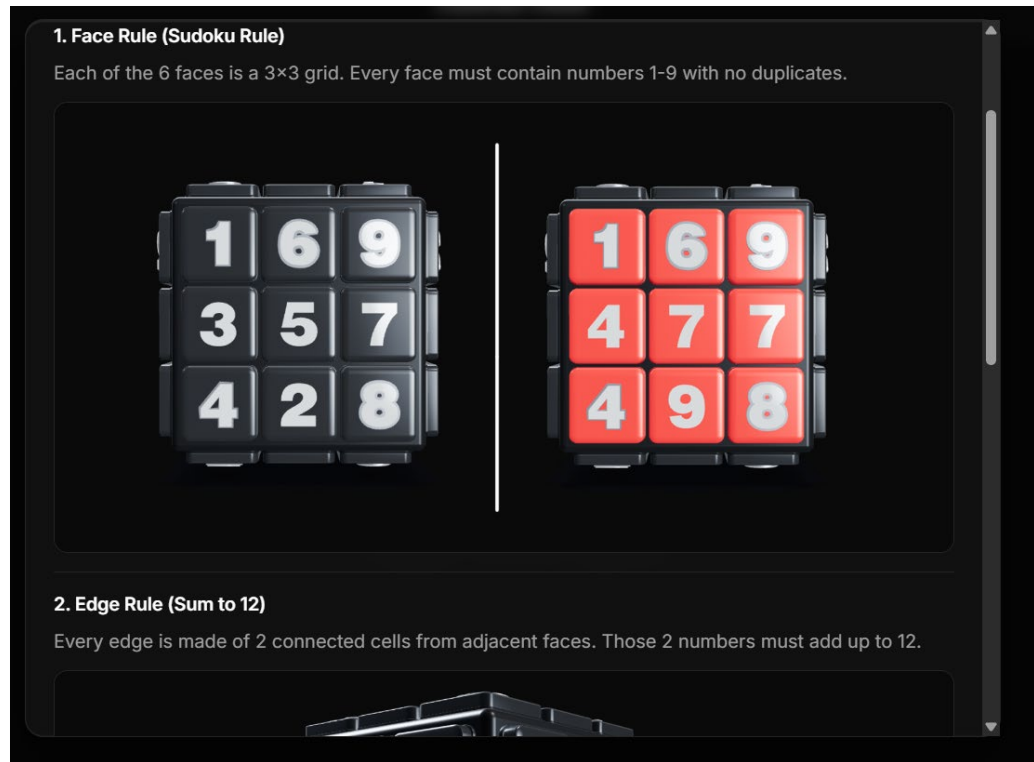
Εικόνα 27: Ο κύβος σε Dark Theme



Εικόνα 28: Παράδειγμα Error State (κόκκινα κελιά)

7.2.2 Προβολή οδηγιών (How to play)

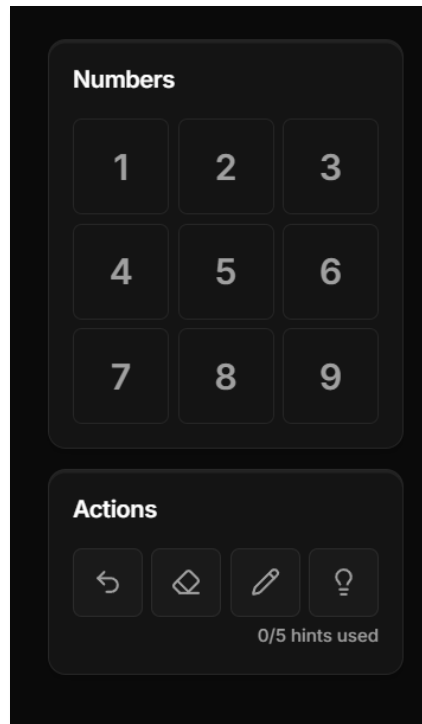
1. Ο χρήστης πατάει το κουμπί "How to Play" από την κεντρική οθόνη του WelcomeModal.
2. Ανοίγει το HowToPlayModal, το οποίο εξηγεί τους τρεις βασικούς κανόνες του παιχνιδιού: Face Rule (Sudoku), Edge Rule (Sum to 12) και Corner Rule (Sum to 12).
3. Ο Client διαβάζει την κατάσταση του ThemeContext (dark ή light mode).
4. Φορτώνονται δυναμικά οι κατάλληλες επεξηγηματικές εικόνες (screenshots) που ταιριάζουν με το τρέχον θέμα του χρήστη.



Εικόνα 29: Κομμάτι του HowToPlayModal

7.2.3 Τοποθέτηση Αριθμού

1. Ο παίκτης επιλέγει έναν αριθμό από το number selector
2. Κάνει click σε κελί του κύβου (3D raycasting με cell identification)
3. Pointer down: press-in animation (GSAP, 0.08s, elastic easing)
4. Click detection: αγνόηση drag (>5px μετακίνηση = rotation, όχι click)
5. Sound effect (pressAudio)
6. Ο Client στέλνει `POST /api/game/move { face, row, column, value, currentState[54], lockedState[54] }`
7. Ο Server κάνει ανακατασκευή Cube, έπειτα lock check, IndividualChecker, re-validate all και τέλος CheckIfSolved
8. Ο Client κάνει ενημέρωση cell states και εκτελεί error animations αν χρειάζεται
9. Γίνεται αυτόματη αποθήκευση προόδου σε localStorage



Εικόνα 30: Number Selector & Actions Buttons

```
// If the pointer moved more than 5px since mousedown, the user
// was rotating the cube – don't place a number.
const native = e.nativeEvent ?? e;
if (pointerDownPos.current) {
  const dx = native.clientX - pointerDownPos.current.x;
  const dy = native.clientY - pointerDownPos.current.y;
  if (Math.sqrt(dx * dx + dy * dy) > 5) return;
}
```

Εικόνα 31: Έλεγχος click εάν είναι για τοποθέτηση αριθμού, ή rotate κύβου

7.2.4 Σημειώσεις (Pencil Mode)

1. Ο παίκτης πατά το pencil icon για να γίνει ενεργοποίηση pencil mode
2. Όταν γίνεται click σε ένα κελί, πραγματοποιείται toggle σημείωσης αριθμού (αντί τοποθέτησης)
3. Οι σημειώσεις εμφανίζονται ως μικρογραφίες (3×3 grid) μέσα στο κελί
4. Κατά την τοποθέτηση κανονικού αριθμού, οι σημειώσεις αφαιρούνται αυτόματα
5. Οι σημειώσεις αποθηκεύονται στο localStorage μαζί με τη λουπή πρόοδο



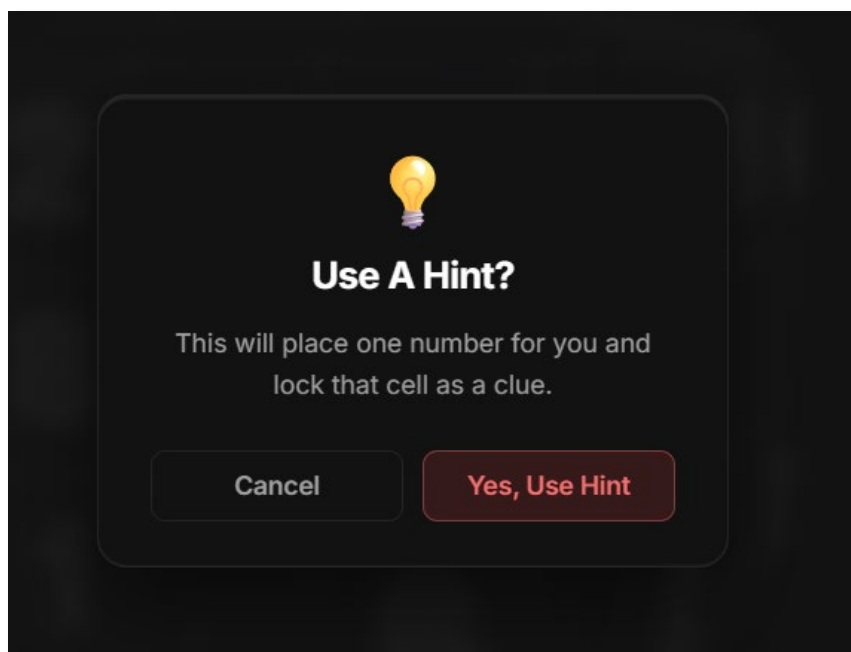
Εικόνα 32: Παραδείγματα Pencil Notes

7.2.5 Αναίρεση (Undo)

1. Όταν click στο undo button
2. Pop τελευταίου move από τη στοίβα undo
3. Ο Client στέλνει `POST /api/game/revert { currentState[54] (μετά undo) }`
4. Ο Server κάνει re-validate ολόκληρου board, και έπειτα `response.IsSolved = false`
5. Ο Client ενημερώνει το board state

7.2.6 Υποδείξεις (Hints)

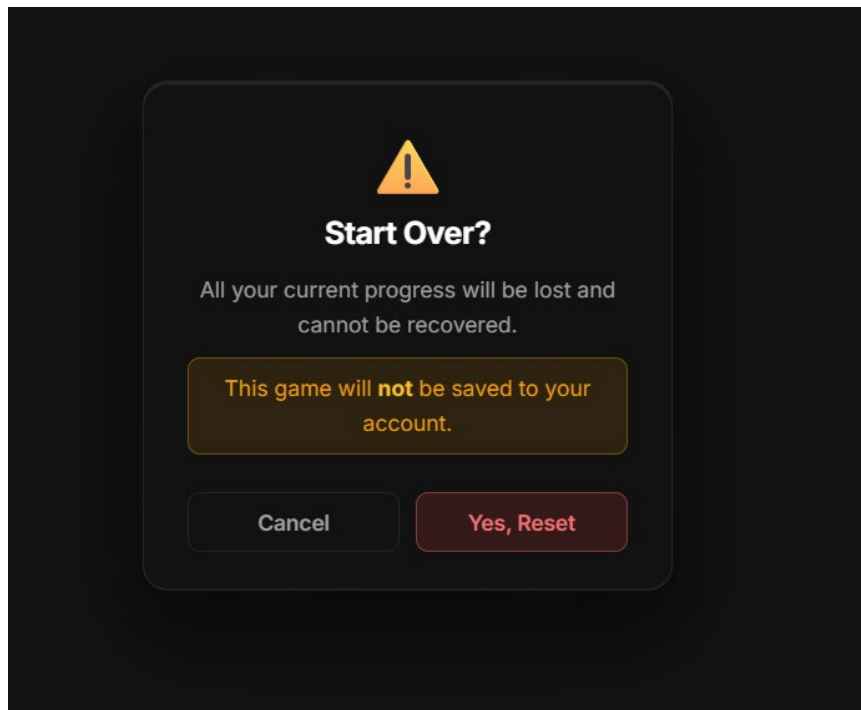
1. Ο παίκτης πατά hint button (εμφανίζεται remaining count, π.χ. "3/5")
2. Αν `hints == 0` τότε shake animation στο button (CSS animation)
3. Ο Client στέλνει `POST /api/game/hint { currentState[54], lockedState[54] }`
4. Ο Server: Φάση 1 (σφάλμα;), Φάση 2 (λογικό βήμα)
5. Στον Client, η κάμερα κάνει animate στην κατάλληλη έδρα, το κελί γίνεται highlighted, αριθμός τοποθετείται



Εικόνα 33: Hint popup

7.2.7 Επανεκκίνηση Γρίφου (Start Over)

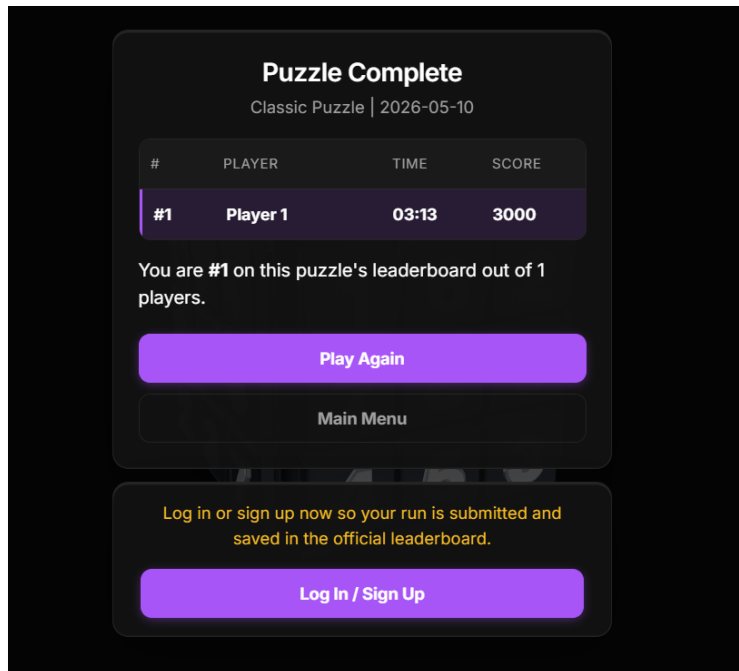
1. Κατά τη διάρκεια του παιχνιδιού, ο παίκτης πατάει το κουμπί "Start Over" (εικονίδιο επαναφοράς) από το μενού εργαλείων.
2. Εμφανίζεται ένα modal επιβεβαίωσης (Start Over?) για την αποφυγή ακούσιας απώλειας της πρόοδου.
3. Εφόσον ο παίκτης επιβεβαιώσει, ο Client διαγράφει την αποθηκευμένη πρόοδο της συγκεκριμένης δυσκολίας από το localStorage.
4. Το χρονόμετρο σταματάει, ο χρόνος μηδενίζεται και όλες οι καταγραφές λαθών (mistakes) και πόντων (score) αρχικοποιούνται.
5. Το boardData επανέρχεται στην αρχική του κατάσταση, χρησιμοποιώντας τον αποθηκευμένο πίνακα lockedCellsRef, διαγράφοντας παράλληλα όλες τις σημειώσεις (pencil notes) του παίκτη.



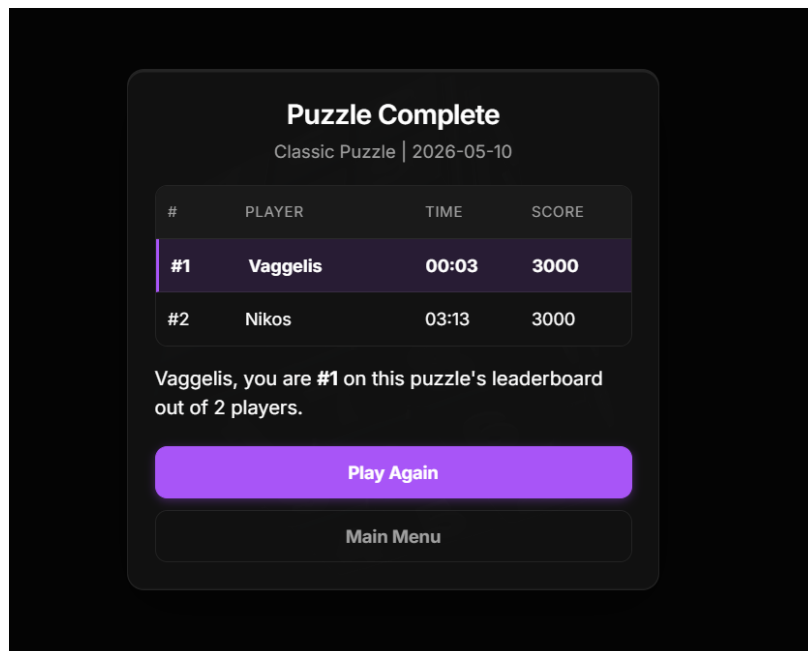
Εικόνα 34: Popup επιβεβαίωσης χρήστη για επανεκκίνηση γρίφου

7.3 Workflow Ολοκλήρωσης

1. Όταν τελευταίο κελί, κάνει `POST /api/game/move`, τότε `isSolved = true`
2. Completion animation (face-by-face, sequenced)
3. Εμφάνιση completion modal: χρόνος, λάθη, hints, σκορ
4. Αν authenticated: `POST /api/user/complete`, ο server αποθηκεύει, τότε leaderboard rank animation
5. Αν guest: `POST /api/user/preview-rank`, κατόπιν review rank (χωρίς αποθήκευση)
6. Leaderboard reveal: animated rank counter + nearby rows table
7. Κουμπιά: "Play again" (νέος γρίφος) / "Return" (WelcomeModal)



Εικόνα 35: Completion Modal, με 1 παίκτη, ως guest



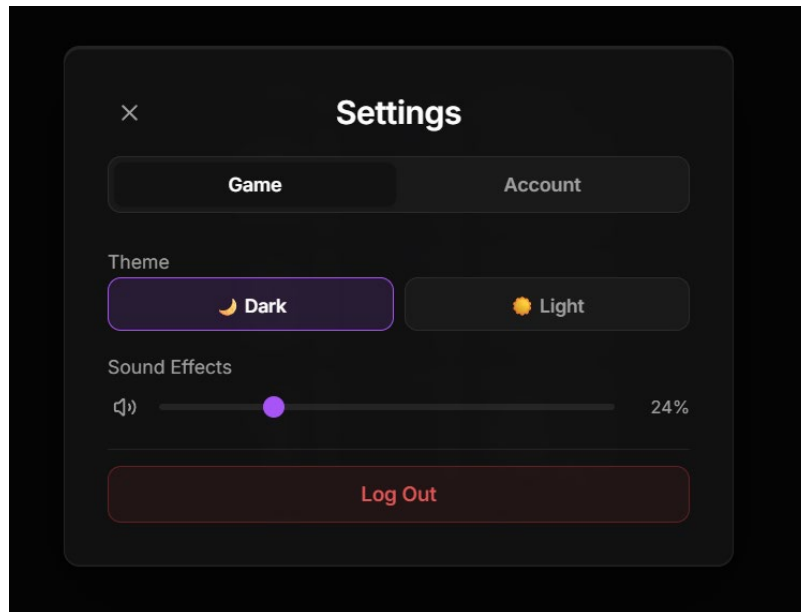
Εικόνα 36: Completion Modal, με 2 παίκτες, ως συνδεδεμένος χρήστης

7.4 Workflow Διαχείρισης & Ρυθμίσεων

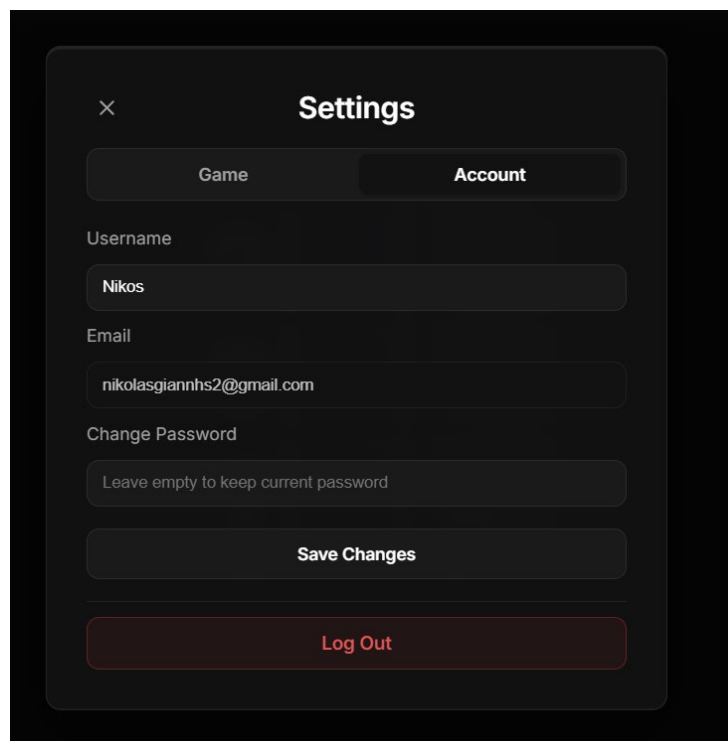
7.4.1 Workflow Ρυθμίσεων (Settings)

1. χρήστης ανοίγει το SettingsModal πατώντας το αντίστοιχο εικονίδιο.
2. Εμφανίζεται το modal με δύο βασικές καρτέλες: "Game" και "Account".
3. Στην καρτέλα "Game", ο χρήστης μπορεί να επιλέξει θέμα (Dark / Light) το οποίο εφαρμόζεται άμεσα μέσω του ThemeContext. Επίσης μπορεί να ρυθμίσει την ένταση του ήχου (Slider), η οποία αποθηκεύεται στο localStorage.
4. Στην καρτέλα "Account", ο συνδεδεμένος χρήστης εισάγει νέο username ή/και νέο password.
5. Με το πάτημα του "Save Changes", εμφανίζεται υποχρεωτικά ένα popup επιβεβαίωσης (Verify Identity).

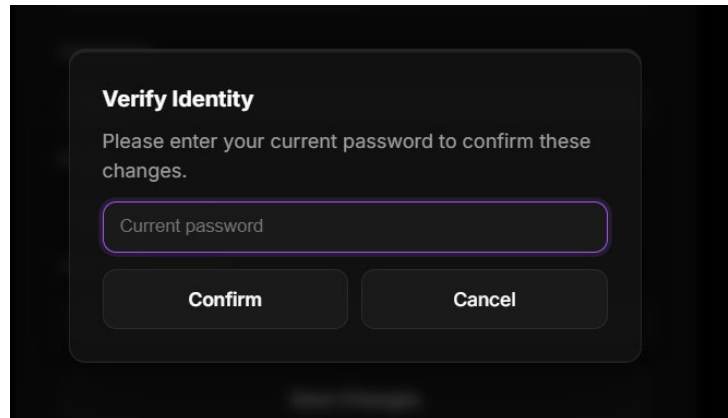
6. Ο χρήστης εισάγει τον τρέχοντα κωδικό του και ο Client στέλνει POST /api/auth/profile/update.
7. Ο Server επαληθεύει τον κωδικό, εφαρμόζει τις αλλαγές και επιστρέφει το νέο JWT token το οποίο αποθηκεύεται στον Client.



Εικόνα 37: SettingsModal στο Game Tab



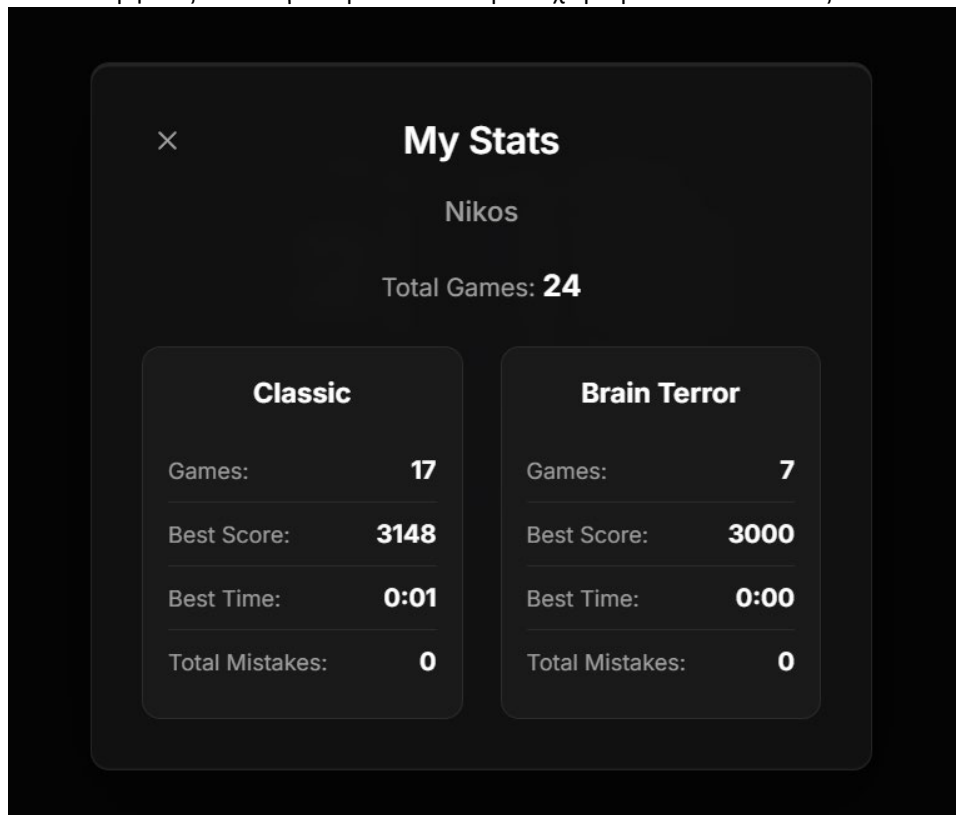
Εικόνα 38: SettingsModal στο Account Tab



Εικόνα 39: Verify Identity popup κατά την αλλαγή στοιχείων λογαριασμού

7.4.2 Workflow Προβολής Στατιστικών (My Stats)

1. Ο συνδεδεμένος χρήστης πατάει "My Stats" από το WelcomeModal.
2. Εμφανίζεται το πάνελ στατιστικών και ο Client κάνει `GET /api/user/stats` κάνοντας χρήση του Bearer Token.
3. Ο Server ανακτά όλα τα GameResult του συγκεκριμένου χρήστη από τη βάση δεδομένων.
4. Υπολογίζονται και ομαδοποιούνται τα δεδομένα: Συνολικά παιχνίδια, στατιστικά για Classic (Αγώνες, Best Score, Best Time, Total Mistakes) και αντίστοιχα για Brain Terror.
5. Ο Client εμφανίζει τα συγκεντρωτικά δεδομένα χωρισμένα σε δυσκολίες.



Εικόνα 40: My Stats Modal

7.5 Workflow Παιχνιδιού

7.5.1 Έναρξη Νέου Γρίφου

Αν υπάρχει αποθηκευμένη πρόοδος στο localStorage:

- WelcomeModal εμφανίζει "Continue — MM:SS" κάτω από κάθε difficulty card
- Click, κάνει φόρτωση boardData, lockedCells, timer, mistakes, notes από localStorage

- Αν η ημερομηνία γρίφου δεν ταιριάζει με σήμερα: η πρόοδος αγνοείται

7.5.2 Συντονισμός Tabs (Tab Sync)

Η εφαρμογή χρησιμοποιεί το **BroadcastChannel API** για συντονισμό μεταξύ tabs:

```
const channel = new BroadcastChannel('cubedoku_tab_sync');
channelRef.current = channel;

channel.onmessage = (event: MessageEvent<TabSyncMessage>) => {
  const msg = event.data;
  if (msg.type === 'GAME_STARTED') takeoverRef.current(msg.difficulty);
  if (msg.type === 'GAME_RECLAIMED') reclaimedRef.current(msg.difficulty);
};
```

Εικόνα 41: Χρήση του BroadcastChannel API

Όταν ένα tab ξεκινά παιχνίδι, στέλνει *GAME_STARTED*, και αυτόματα τα άλλα tabs εμφανίζουν "paused" overlay και σταματούν τον timer.

7.5.3 Themes (Dark/Light)

- Αρχικό θέμα: system preference (prefers-color-scheme)
- Toggle μέσω ThemeContext, εφαρμογή μέσω data-theme attribute στο <html>
- CSS variables (index.css) αλλάζουν αυτόματα
- 3D materials: animated μετάβαση μέσω GSAP (tweenMatDef)
- Difficulty card images: αλλαγή ανάλογα με θέμα



Εικόνα 42: Σύγκριση Dark Theme με Light Theme

7.5.4 Animation & Ήχοι

Για τη βελτίωση της αίσθησης αλληλεπίδρασης, κάθε κελί διαθέτει **press animation** στις παρακάτω φάσεις:

- κατά το pointer-down, το κελί βυθίζεται προς το εσωτερικό του κύβου κατά 0.09 μονάδες στον κατάλληλο άξονα (ανάλογα με την έδρα) με GSAP tween διάρκειας 0.08s.
- Κατά το pointer-up, επιστρέφει στην αρχική θέση με elastic easing (0.35s), δημιουργώντας μια φυσική αίσθηση ελατηρίου.
- Τα κελιά που εισέρχονται σε κατάσταση σφάλματος εκτελούν ένα στιγμιαίο **nudge**

animation προς τα έξω (0.03 μονάδες) με γογο repeat, προσελκύοντας την προσοχή του παίκτη χωρίς να διακόπτουν τη ροή του.

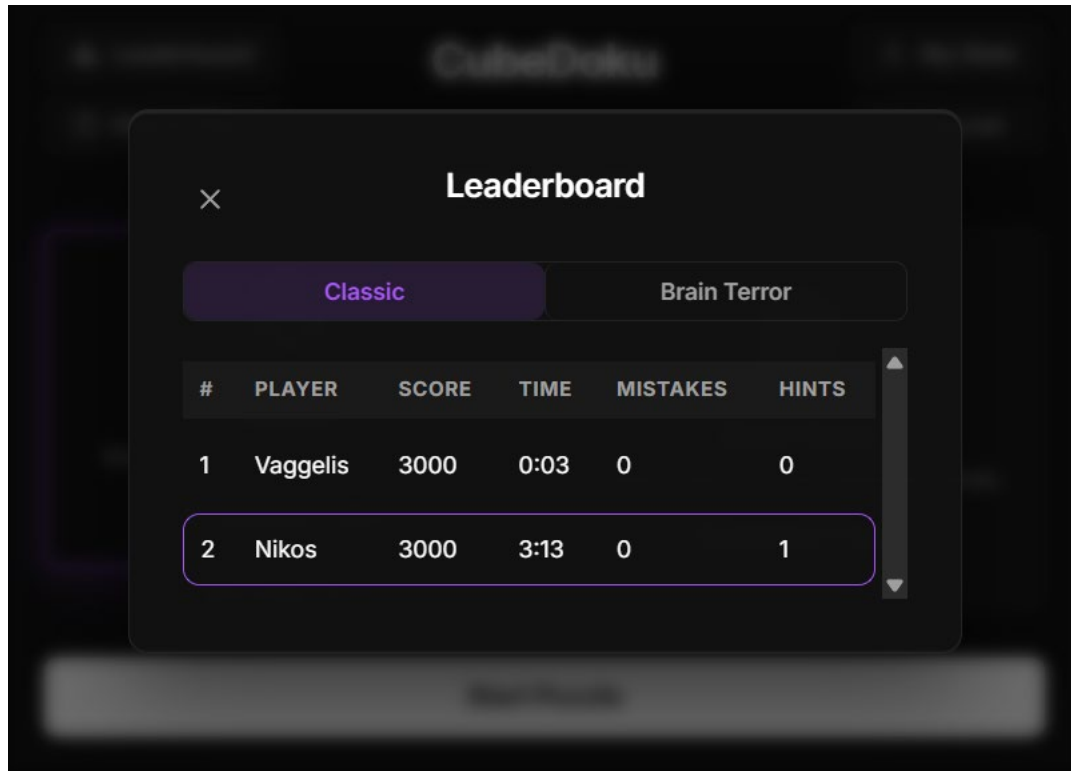


Εικόνα 43: Πατημένο κελί στον κύβο

Επιπλέον η εφαρμογή ενσωματώνει δύο ηχητικά εφέ: έναν ήχο τοποθέτησης (press) και έναν ήχο σφάλματος (error). Τα αντίστοιχα αντικείμενα Audio δημιουργούνται ως module-level singletons στο αρχείο `audioManager.ts`, ώστε η ρύθμιση έντασης να εφαρμόζεται κεντρικά και η ένταση να διατηρείται μεταξύ συνεδριών μέσω `localStorage`.

7.6 Workflow Προβολής Κατάταξης (Leaderboard)

1. Ο χρήστης πατάει το κουμπί "Leaderboard" από το `WelcomeModal` ή κατά την διάρκεια του παιχνιδιού
2. Εμφανίζεται το `LeaderboardModal` και ο Client κάνει `GET /api/user/leaderboard?date=YYYY-MM-DD` (για τη σημερινή ημερομηνία).
3. Ο Server ανακτά και ταξινομεί όλα τα σημερινά αποτελέσματα βάσει του Score και του χρόνου, και τα επιστρέφει στον Client.
4. Ο Client φιλτράρει τα αποτελέσματα τοπικά, ανάλογα με την επιλεγμένη καρτέλα δυσκολίας (Classic ή Brain Terror).
5. Η κατάταξη εμφανίζεται σε μορφή πίνακα (Rank, Player, Score, Time, Mistakes, Hints).
6. Το σκορ του τρέχοντος παίκτη επισημαίνεται χρωματικά (highlight) εφόσον υπάρχει ταύτιση στο όνομα χρήστη.



Εικόνα 44: Leaderboard Modal

8. Στρατηγική Ελέγχου (Testing & QA)

8.1 Μεθοδολογία Ελέγχου

Η εφαρμογή ελέγχθηκε μέσω συνδυασμού μεθόδων:

8.1.1 Manual Testing

- **Functional testing:** Κάθε workflow χρήστη δοκιμάστηκε manually. Εγγραφή, σύνδεση, gameplay, completion, leaderboard
- **Cross-browser testing:** Brave, Chrome, Firefox, Edge. Δώθηκε ιδιαίτερη προσοχή στο WebGL rendering
- **Responsive testing:** Desktop resolutions, tablet viewport
- **Edge cases:** πολλαπλά tabs, expired tokens, network failures, empty puzzle states

8.1.2 Algorithmic Verification

Οι αλγόριθμοι δημιουργίας γρίφων ελέγχθηκαν εντατικά:

- **Μοναδικότητα λύσης:** Κάθε γεννημένος γρίφος ελέγχεται αυτόματα μέσω CountSolutions(cube, 2). Αν βρεθούν ≥ 2 λύσεις, ο γρίφος απορρίπτεται.
- **Λογική επιλυσιμότητα:** Ο LogicalSolver επαληθεύει ότι ο γρίφος μπορεί να λυθεί χωρίς backtracking (αλλιώς δεν θα ήταν fair για τον παίκτη)
- **Δυσκολία ταξινόμηση:** Η δυσκολία επαληθεύεται αυτόματα βάσει τεχνικών. Εάν απαιτούνται μόνο Naked/Hidden Singles πάμε σε Classic, αλλιώς σε BrainTerror

8.1.3 Security Testing

- **Input validation:** Δοκιμή αρνητικών τιμών, υπερβολικά μεγάλων strings, SQL injection attempts μέσω DataAnnotations
- **Authentication:** Δοκιμή expired tokens, invalid tokens, missing Authorization header
- **Rate limiting:** Επαλήθευση 429 responses μετά από υπέρβαση limits
- **CORS:** Επαλήθευση rejection requests από μη-εγκεκριμένα origins

8.2 Development-Only Tools

8.2.1 Solution Endpoint

Το `GET /api/game/solution` endpoint επιστρέφει την πλήρη λύση. Χρησιμοποιήθηκε κατά την ανάπτυξη του frontend για debugging rendering:

```
// GET /api/game/solution?difficulty=Classic|BrainTerror
// Returns the full solved puzzle (all 54 cells) || DEV ONLY
// TODO: Remove on production
[HttpGet("solution")]
0 references
public ActionResult<List<CellDto>> GetSolution([FromQuery] string difficulty = "Classic")
{
    if (!env.IsDevelopment())
        return NotFound();

    lock (_lock)
    {
        var solution = difficulty == "Classic" ? _classicSolution : _brainterrorSolution;
        if (solution == null)
            return BadRequest("No puzzle generated yet. Call /api/game/start first.");
        return Ok(solution);
    }
}
```

Εικόνα 45: DEV-Solution function

8.2.2 Console Logging

Ο PuzzleGenerator εκτυπώνει αναλυτικές πληροφορίες στο console, για βοήθεια κατά την διάρκεια του developing:

- Αριθμός iterations του solver
- Πόσα κελιά αφαιρέθηκαν
- Ποιες τεχνικές χρησιμοποίησε ο LogicalSolver
- Βήμα-βήμα λύση

8.3 Ασφάλεια – Defense in Depth

8.3.1 Rate Limiting Policies

Για την ανάπτυξη των rate limiting policies έγινε χρήση Generative AI ώστε να μην υπάρχουν κενά ασφαλείας.

Policy	Limit	Window	Endpoint
AuthPolicy	10 requests	1 minute	Register, login, google
CompletionPolicy	5 requests	1 hour	Complete
GlobalPolicy	120 requests	1 minute	All others

Πίνακας 5: Policies και οι ιδιότητες τους

8.3.2 Security Headers

Για την υλοποίηση των Security headers έγινε χρήση Generative Ai ώστε να μην δημιουργηθεί κάποιο λάθος.

```
// prevent the page from being embedded in an iframe (stops clickjacking attacks)
headers["X-Frame-Options"] = "DENY";

// only send origin information to same-site requests
headers["Referrer-Policy"] = "strict-origin-when-cross-origin";

// disable browser APIs the app doesn't use (camera, mic, location, payment)
headers["Permissions-Policy"] = "camera=(), microphone=(), geolocation=(), payment=()";

// Content Security Policy
// default-src 'self': only load resources from the same origin by default
// script-src 'self': only execute scripts from same origin (no CDN scripts)
// style-src 'unsafe-inline': needed for React's styled components / class injection
// Google fonts: allowed because we use them for typography
// connect-src: API calls go to 'self', Google OAuth goes to accounts.google.com
headers["Content-Security-Policy"] =
  "default-src 'self'; " +
  "script-src 'self'; " +
  "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; " +
  "font-src 'self' https://fonts.gstatic.com; " +
  "img-src 'self' data: https:; " +
  "connect-src 'self' https://accounts.google.com https://www.googleapis.com; " +
  "frame-ancestors 'none';";
```

Εικόνα 46: Security headers

8.3.3 Server-side Lock Check

Ακόμα κι αν ο client δεν στέλνει modification σε locked κελί, ο server ελέγχει:

```
if (request.LockedState != null && request.LockedState.Length == 54)
{
    int targetIndex = GetCellIndex(targetFace, request.Row, request.Column);
    if (request.LockedState[targetIndex])
    {
        return BadRequest("Cannot modify locked clue cell.");
    }
}
```

Εικόνα 47: Security headers

8.4 Γνωστοί Περιορισμοί

- Η βαθμολογία υπολογίζεται client-side. Αυτό έχει πιθανότητα να γίνει abused, περιορίζεται όμως με server-side clamping.
- Δεν υπάρχει token revocation/blacklisting, ωστόσο δεσμεύεται με 60-min expiry
- Δεν υπάρχουν database-level unique constraints, όλα εφαρμόζονται σε application level
- Η *unsafe-inline* στο CSP για styles είναι αναγκαία λόγω React runtime styling

9. Συμπεράσματα & Μελλοντικές Επεκτάσεις

9.1 Σύνοψη Επιτευγμάτων

Η παρούσα πτυχιακή παρουσίασε τον σχεδιασμό και την υλοποίηση του CubeDoku, μιας ολοκληρωμένης εφαρμογής παζλ λογικής που επεκτείνει το κλασικό Sudoku σε τρεις διαστάσεις. Τα κύρια επιτεύγματα περιλαμβάνουν:

- **Μαθηματική Θεμελίωση:** Σχεδιάστηκε ένα σύνολο κανόνων (face rule, edge 12-sum, corner 12-sum) που εξασφαλίζει τη δημιουργία μαθηματικά ορθών γρίφων μοναδικής λύσης. Η τιμή-στόχος 12 αποδεικνύεται βέλτιστη για τη γεωμετρία κύβου 3×3 με τιμές 1-9.
- **Αλγοριθμική Αρτιότητα:** Υλοποιήθηκε backtracking solver με MRV ευρετική, logical solver με τέσσερις τεχνικές ανθρώπινης λογικής, και στρατηγικός puzzle generator που εγγυάται μοναδική λύση και ελεγχόμενη δυσκολία. Η στρατηγική αφαίρεσης Center σε Edge σε Corner βελτιώνει σημαντικά τον ρυθμό επιτυχούς δημιουργίας.
- **Immersive Εμπειρία:** Η τρισδιάστατη απεικόνιση μέσω React Three Fiber/Three.js, σε συνδυασμό με GSAP animations, IBL lighting, theme-aware materials, και ηχητικά εφέ,

δημιουργεί μια premium αίσθηση παιχνιδιού.

- **Ασφαλής Αρχιτεκτονική:** Η πολυεπίπεδη ασφάλεια (JWT, BCrypt, rate limiting, security headers, input validation, anti-enumeration, CORS) παρέχει ένα production-ready σύστημα.
- **Ολοκληρωμένη Εμπειρία Gaming:** Ημερήσιος γρίφος, σύστημα βαθμολογίας, ανταγωνιστικό leaderboard, λογικές υποδείξεις, διατήρηση προόδου, cross-tab sync, και dual-theme support.

9.2 Μελλοντικές Επεκτάσεις

9.2.1 Βραχυπρόθεσμες

- **Database-level unique constraints:** Μετάβαση από application-level σε DB-level enforcement (UserId + Difficulty + PuzzleDate)
- **JwtService refactoring:** Μεταφορά GenerateJwt σε ξεχωριστή service class
- **Token refresh mechanism:** Αυτόματη ανανέωση tokens χωρίς re-login
- **Server-side score computation:** Αφαίρεση δυνατότητας client-side manipulation

9.2.2 Μεσοπρόθεσμες

- **Multiplayer mode:** Real-time αγώνες για λύση του ίδιου κύβου, μέσω πιθανόν WebSocket
- **Weekly/Monthly leaderboards:** Πίνακες κατάταξης μεγαλύτερου χρονικού ορίου
- **Achievement system:** Badges για milestones (100 games, perfect score, κ.λπ.)
- **Tutorial mode:** Αλληλεπιδραστικό εκπαιδευτικό παιχνίδι πέρα από το 'How to Play' modal
- **Mobile responsive design:** Βελτιστοποίηση touch interactions στην Mobile μορφή

9.2.3 Μεσοπρόθεσμες

- **React Native/Flutter mobile app:** Native εφαρμογή iOS/Android λογισμικών
- **Cube size variations:** 4×4 ή 5×5 πλέγματα ανά έδρα
- **Social features:** Friends, challenges, shared puzzles
- **Accessibility:** Screen reader support, color-blind modes

9.3 Επίλογος

Η ανάπτυξη του CubeDoku απέδειξε ότι η μεταφορά κλασικών 2D παζλ σε τρισδιάστατο χώρο μπορεί να δημιουργήσει νέες, ενδιαφέρουσες προκλήσεις τόσο σε αλγοριθμικό όσο και σε σχεδιαστικό επίπεδο. Η γεωμετρία του κύβου εισάγει φυσικούς περιορισμούς (ακμές, γωνίες) που εμπλουτίζουν τη λογική του παζλ χωρίς τεχνητή πολυπλοκότητα. Ταυτόχρονα, οι σύγχρονες τεχνολογίες web (WebGL, React, ASP.NET Core) καθιστούν δυνατή τη δημιουργία premium gaming εμπειριών απευθείας στον browser, χωρίς την ανάγκη για εγκατάσταση εφαρμογής.

Βιβλιογραφία

- [1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.), Chapter CSP, Backtrack MRV Heuristic. Pearson.
- [2] Simonis, H. (2005). "Sudoku as a Constraint Problem." *CP Workshop on Modeling and Reformulating CSPs*.
- [3] OWASP Foundation. (2021). *OWASP Top 10 Web Application Security Risks*.
- [4] Microsoft. (2024). ASP.NET Core Documentation. <https://learn.microsoft.com/aspnet/core>
- [5] Three.js Documentation. <https://threejs.org/docs>
- [6] React Three Fiber Documentation. <https://docs.pmnd.rs/react-three-fiber>
- [7] GreenSock. *GSAP Documentation*. <https://gsap.com/docs>
- [8] PostgreSQL Documentation. <https://www.postgresql.org/docs>
- [9] Auth0. (2023). *Introduction to JSON Web Tokens*. <https://jwt.io/introduction>
- [10] Provenza, J. (2012). *BCrypt Hashing Algorithm*.